Neural Networks 206 (2027) 109832

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Review

Monotonicity constraints in neural networks: A survey and future directions

Junpeng Sun [a], Juan Luis Suárez [a], Weiping Ding [b,c,*], Salvador García [a]

[a] Department of Computer Science and Artificial Intelligence, Andalusian Research Institute in Data Science and Computational Intelligence, University of Granada, Granada, 18071, Spain
[b] School of Artificial Intelligence and Computer Science, Nantong University, Nantong, 226019, China
[c] Faculty of Data Science, City University of Macau, Macau, 999078, China

ARTICLE INFO

Keywords:
Trustworthy machine learning
Neural network
Domain knowledge
Monotonicity constraint
Deep learning

ABSTRACT

Monotonicity constraints are important in many machine learning applications where domain knowledge indicates that predictions should follow monotonic relationships with respect to certain input features. Incorporating monotonicity constraints into neural networks has attracted increasing attention. However, existing neural networks differ significantly in how they implement monotonicity and lack a systematic analysis of monotonicity design. This paper provides a comprehensive study of monotonicity constraints in neural networks. First, we formalize monotonicity and clarify its relationship to related concepts. Based on this, we propose a unified taxonomy that organizes existing methods into two categories: structure-based monotonicity approaches, which guarantee monotonicity through architectural design and parameter constraints, and non-structural monotonicity approaches, which induce, verify, or certify monotonicity without encoding it directly into the architecture. Second, we conduct a systematic empirical comparison of representative methods across 12 real-world tabular datasets using a unified evaluation protocol. The results show that carefully designed structure-based approaches can achieve predictive performance comparable to unconstrained neural networks while guaranteeing monotonic behavior, whereas naive constraint implementations often suffer from limited expressive capacity. Non-structural approaches provide greater flexibility and can adapt to existing architectures, but their effectiveness and guarantee strength depend strongly on the specific mechanism. Overall, this work offers a unified perspective on designing monotonicity constraints in neural networks. The source code and datasets are available at https://github.com/junpeng-sun/MonNN.

1. Introduction

Over the past decade, advancements in neural networks have profoundly transformed artificial intelligence (Abdi et al., 1999; Mari & Snidaro, 2026; Tang et al., 2024), enabling the development of powerful predictive models that support data-driven decision-making across diverse domains, including fraud detection, cybersecurity, medical diagnostics, and financial forecasting (Capstick et al., 2025; Chen et al., 2003; Ding et al., 2022; Motie & Raahemi, 2024; Pawlicki et al., 2022).

As neural networks are increasingly applied to high-impact and socially sensitive decision-making scenarios, the focus has shifted beyond prediction performance itself (Barredo Arrieta et al., 2020; Kasongo, 2023; Kumar Jain et al., 2024; Li et al., 2026, 2023; Wang et al., 2025a; Zhao et al., 2024b). Some models, while highly accurate, exhibit unintuitive or contradictory behavior, undermining trust and complicating deployment. To address these issues, researchers are increasingly emphasizing the integration of domain knowledge and structural constraints into neural network models (Dash et al., 2022; Goyal et al., 2024; Roychowdhury et al., 2021; Stewart & Ermon, 2017). Among various forms of structural constraints, monotonicity is particularly important (Chatterjee & Mukherjee, 2019; González-Almagro et al., 2024; Khalaf et al., 2026; Suárez et al., 2024; Wang et al., 2020). It expresses an intuitive requirement: the model's output should change consistently with respect to certain input features. For instance, higher income should not increase estimated credit risk, more severe clinical indicators should not decrease predicted disease severity, and stronger safety signals should not lower the system reliability score. Even if a model's overall predictive performance appears competitive, violating these expectations can severely damage trust. Therefore, monotonicity has become a key factor in developing trustworthy machine learning systems that align with human cognition (Cano & García, 2017; Hurault & Marques-Silva, 2023; Polo-Molina et al., 2026a).

While the monotonicity of neural networks is crucial, ensuring it remains a significant challenge (Jazbec et al., 2023; Yanaka et al., 2019;

* Corresponding author.
E-mail addresses: junpengsunx@correo.ugr.es (J. Sun), jlsuarezdiaz@decsai.ugr.es (J.L. Suárez), dwp9988@163.com (W. Ding), salvagl@decsai.ugr.es (S. García).

https://doi.org/10.1016/j.neunet.2026.109832
Received 24 April 2026; Received in revised form 19 August 2026; Accepted 29 October 2026
Available online 2 November 2026
0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Zhao et al., 2024a). When trained solely to minimize empirical loss, neural networks may learn highly counterintuitive decision functions, especially in sparse data, under noisy supervision, or when extrapolation exceeds the training distribution. These problems are exacerbated in complex, high-dimensional environments, as unconstrained models may exploit spurious correlations that contradict known monotonic relationships. Consequently, ensuring the monotonicity of neural networks requires explicit mechanisms that go beyond standard training objectives. This challenge is particularly pronounced for neural networks because their high flexibility allows them to fit complex patterns, but also increases the risk of learning spurious, non-monotonic relationships under unconstrained conditions.

From a learning theory perspective, introducing monotonic constraints can be viewed as a principled inductive bias. By restricting the hypothesis space to functions that satisfy predefined monotonic relations, models can achieve better generalization, stronger robustness to noise, and greater resistance to adversarial perturbations. However, monotonicity constraints inevitably reduce model flexibility; when set incorrectly or overly strictly, they may limit the model's expressive power and degrade its predictive performance. Understanding and managing this trade-off between monotonicity enforcement and the expressive power of neural network models is central to our research.

A diverse range of approaches has been proposed to address monotonicity in neural networks. These methods include enforcing monotonic behavior through structural constraints (such as non-negative weights or special lattice constructions) (Daniels & Velikova, 2010; Nguyen et al., 2023; Silva et al., 2018); penalizing behaviors that violate monotonicity during training (Gupta et al., 2019; Liu et al., 2020; Monteiro et al., 2021); and verification-based techniques that assess monotonic behavior post-training (Bunel et al., 2018; Liu et al., 2020; Sivaraman et al., 2020). Additional developments have explored partial monotonicity, where only a subset of input variables is constrained, as well as connections between monotonicity and related properties such as Lipschitz continuity and robustness (Anil et al., 2019; Cooper et al., 2026; Nolte et al., 2023).

However, current research on the monotonicity of neural networks remains fragmented. Specifically, existing studies often focus on isolated methodologies, lacking systematic taxonomies or empirical comparisons across datasets and architectures. Moreover, while the trade-off between predictive performance and monotonicity enforcement is frequently mentioned, rigorous comparative analyses are rarely conducted.

This paper aims to address these limitations by providing a comprehensive survey, taxonomy, and comparative analysis of monotonicity constraints in neural networks. Unlike previous investigations that focused on specific monotonic neural network architectures, this paper adopts a constraint-centric perspective. It systematically categorizes existing methods by how they implement monotonicity in neural networks. Building upon this taxonomy, we conduct extensive empirical evaluations using a unified experimental protocol to clarify the trade-offs between performance and monotonicity for representative methods. The main contributions of this paper are summarized below:

- We present a comprehensive taxonomy of monotonicity constraints in neural networks, organizing existing approaches by how monotonicity is imposed and clarifying the conceptual relationships among structure-based, training-based, and verification- and certification-based monotonicity mechanisms.
- We systematically analyze monotonicity enforcement strategies from both theoretical and architectural perspectives, with particular emphasis on their monotonicity guarantees, expressive capacity, scalability, and applicability to total or partial monotonicity settings, thereby clarifying the trade-offs between different monotonicity designs.
- We establish a unified evaluation protocol for extensive empirical comparison across 12 real-world benchmark datasets, enabling consistent evaluation of representative methods in terms of predictive performance, model complexity, and performance-monotonicity trade-offs.
- We provide practical guidance for selecting monotonicity-constraint mechanisms in neural networks for different application requirements and identify open challenges and future directions for robust, scalable, and application-driven monotonicity design.

This paper aims to promote a more systematic understanding of the monotonicity constraints of neural networks and their role in trustworthy machine learning by integrating scattered research findings into a coherent framework. The survey not only synthesizes existing methods but also highlights their conceptual foundations, practical applications, and empirical performance under a unified evaluation environment. Fig. 1 provides an overview of the survey structure.

This paper focuses specifically on monotonicity constraints in neural networks, emphasizing the mechanisms by which monotonicity is enforced, induced, or verified in neural models. Rather than providing a broader overview of constrained learning, we concentrate on the theoretical foundations, architectural strategies, empirical performance, and practical trade-offs of incorporating monotonicity into neural networks. We therefore do not aim to comprehensively cover ordinal regression, ranking methods, or monotonic modeling in non-neural frameworks, except where they provide concepts directly relevant to monotonicity in neural networks.

The remainder of this paper is organized as follows. Section 2 introduces the theoretical background and fundamental concepts related to monotonicity in neural networks. Section 3 presents a comprehensive taxonomy of monotonicity constraints in neural network approaches, categorizing existing methods according to how monotonicity is enforced. Section 4 provides an extensive experimental evaluation, comparing representative monotonic modeling strategies and analyzing the trade-offs between predictive performance and satisfaction of monotonicity. Section 5 discusses the main challenges of incorporating monotonicity constraints into neural networks and outlines several promising directions for future research. Finally, Section 6 concludes the paper.

2. Theoretical background and fundamental concepts

This section introduces the fundamental definitions and concepts of monotonicity in neural networks.

2.1. Background and definitions

This section formalizes the notion of monotonicity in learning problems, including total and partial monotonicity, and its interpretation in regression and classification settings. There is a dataset $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ that consists of input-output pairs drawn from an unknown joint distribution. The input vector $\mathbf{x} \in \mathcal{X} \subseteq \mathbb{R}^d$ represents a d-dimensional feature space, while the output $y \in \mathcal{Y}$ may be binary, categorical, ordinal, or continuous, depending on the learning task. A learning algorithm aims to estimate a function $f : \mathcal{X} \to \mathcal{Y}$ that minimizes an expected risk with respect to a loss function $\ell(\cdot, \cdot)$, typically through empirical risk minimization. In neural networks, the function f is parameterized by a set of weights θ and optimized using gradient-based methods.

In many real-world applications, domain knowledge imposes directional constraints on the relationship between inputs and outputs (Kaplan et al., 2023; Petersen et al., 2021; Silvestri et al., 2021). One of the most common constraints is monotonicity, which requires that increasing (or decreasing) certain input variables not lead to contradictory changes in the predicted outcome. Formally, given two input vectors $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^d$, we say that $\mathbf{x}$ dominates $\mathbf{x}'$, denoted by $\mathbf{x} \succeq \mathbf{x}'$, if $x_j \geq x'_j, \quad \forall j = 1, \ldots, d$. This dominance relation induces a partial order on the input space.

Definition 1 (Sill & Abu-Mostafa, 1997). A function $f : \mathbb{R}^d \to \mathbb{R}$ is said to be monotonic non-decreasing if

$$\mathbf{x} \succeq \mathbf{x}' \;\Rightarrow\; f(\mathbf{x}) \geq f(\mathbf{x}'). \tag{1}$$

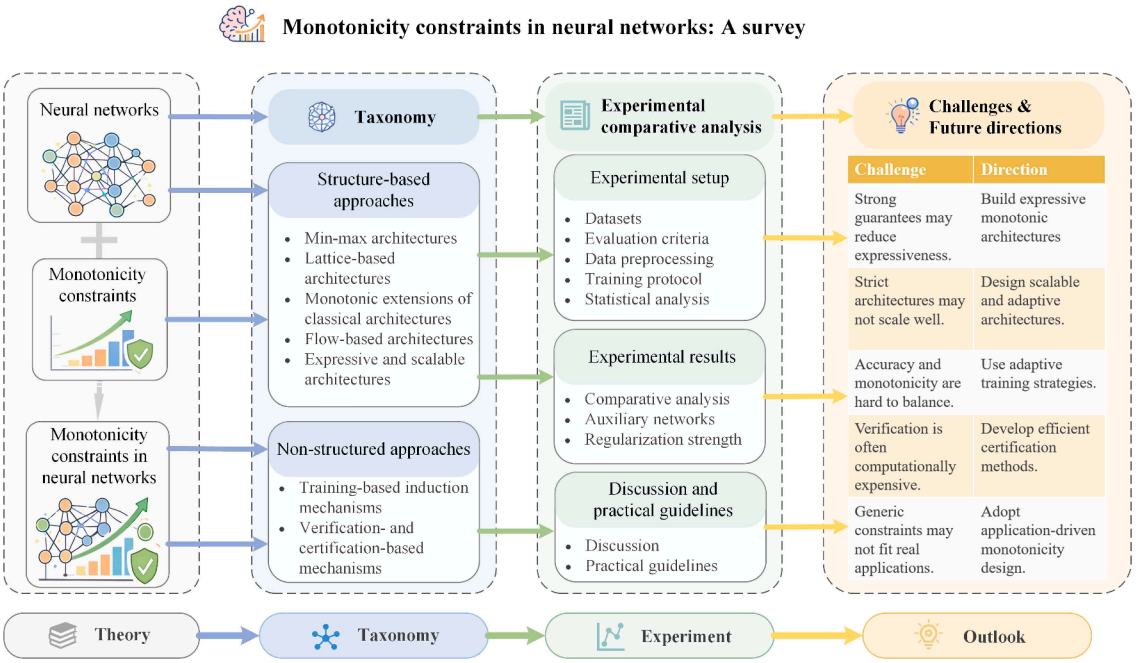


Fig. 1. Overall framework of this survey.

Analogously, f is monotonic non-increasing if $\mathbf{x} \succeq \mathbf{x}'$ implies $f(\mathbf{x}) \leq f(\mathbf{x}')$. Without loss of generality, decreasing monotonicity can be transformed into increasing monotonicity by negating the corresponding input variables.

In practice, monotonicity constraints are often meaningful only for a subset of input features rather than for all dimensions.

Definition 2 (Daniels & Velikova, 2010). Let $\mathcal{M} \subseteq \{1, \ldots, d\}$ denote the indices of features for which monotonic relationships are assumed. A function f satisfies partial monotonicity with respect to $\mathcal{M}$ if

$$x'_j \geq x_j,\ \forall j \in \mathcal{M}, \quad x'_k = x_k,\ \forall k \notin \mathcal{M} \ \Rightarrow\ f(\mathbf{x}') \geq f(\mathbf{x}). \tag{2}$$

If $\mathcal{M} = \{1, \ldots, d\}$, then partial monotonicity reduces to total monotonicity. Partial monotonicity is especially relevant in real applications, since enforcing monotonic constraints on all input dimensions is often unrealistic or overly restrictive.

Monotonicity can be defined for both regression and classification problems. In regression tasks with real-valued outputs, monotonicity is directly expressed through the ordering of predicted values. In classification settings, especially when class labels possess an inherent order, monotonicity is more naturally formulated in terms of conditional class probabilities. Let $P(y \mid \mathbf{x})$ denote the conditional distribution of labels given the input. A commonly adopted notion is stochastic dominance, according to which dominance in the input space implies dominance in the corresponding conditional output distributions (Kotlowski & Slowinski, 2013):

$$\mathbf{x} \succeq \mathbf{x}' \ \Rightarrow\ P(y \geq k \mid \mathbf{x}) \geq P(y \geq k \mid \mathbf{x}'), \quad \forall k, \tag{3}$$

where k ranges over the thresholds of the ordered label set.

For classifiers with ordered outputs, monotonicity can be naturally defined by requiring that dominance in the input space implies stochastic dominance in the output distribution. This formulation is particularly natural in ordinal classification, where class labels admit an ordered structure, and, in the binary case, reduces to requiring monotonicity of the positive-class probability. For unordered multi-class classification, however, monotonicity is generally less direct and typically requires an additional probabilistic or semantic interpretation of the output classes.

Not all pairs of input instances are comparable under the dominance relation. Two inputs $\mathbf{x}$ and $\mathbf{x}'$ are said to be comparable if either $\mathbf{x} \succeq \mathbf{x}'$ or $\mathbf{x}' \succeq \mathbf{x}$. A dataset is said to be monotonicity-consistent if every comparable pair satisfies the corresponding ordering in the outputs. In practice, real-world datasets often contain violations of monotonicity due to noise, measurement errors, annotation uncertainty, or unobserved confounding factors. As a consequence, exact monotonic consistency may be unrealistic at the data level, motivating learning paradigms that allow soft constraint enforcement or controlled violations.

For differentiable models, monotonicity can also be characterized through partial derivatives. Let $\mathcal{X}$ be coordinatewise convex in the direction of feature x_j. A sufficient condition for f to be non-decreasing with respect to x_j throughout $\mathcal{X}$ is

$$\frac{\partial f(\mathbf{x})}{\partial x_j} \geq 0, \qquad \forall \mathbf{x} \in \mathcal{X}. \tag{4}$$

Similarly, $\partial f(\mathbf{x})/\partial x_j \leq 0$ for every $\mathbf{x} \in \mathcal{X}$ is sufficient for non-increasing behavior. A derivative-sign condition evaluated only at isolated points provides a local empirical diagnostic rather than a domain-wide monotonicity guarantee.

Monotonicity is closely related to other desirable properties of learning models, particularly robustness and stability. A commonly used notion to quantify stability is Lipschitz continuity (Chen et al., 2026; Gouk et al., 2021). A function $f : \mathbb{R}^d \to \mathbb{R}$ is Lipschitz continuous if there exists a constant $L > 0$ such that

$$|f(\mathbf{x}) - f(\mathbf{x}')| \leq L \cdot \|\mathbf{x} - \mathbf{x}'\|, \quad \forall \mathbf{x}, \mathbf{x}'. \tag{5}$$

Bounding the Lipschitz constant limits the sensitivity of model outputs to small input perturbations, thereby improving robustness to noise and adversarial perturbations. Although Lipschitz continuity and monotonicity are distinct properties, they are often considered together in constrained neural network design: monotonicity controls the directional behavior of predictions with respect to selected inputs, whereas Lipschitz continuity controls the magnitude of output variation. Their combination can promote models that are both behaviorally consistent and stable under perturbations.

2.2. Properties and constraints related to monotonicity

Monotonicity is closely related to several structural properties and learning frameworks studied in machine learning and neural network

theory, including Lipschitz continuity, convexity, smoothness, differentiability, and isotonic regression.

- **Monotonicity versus Lipschitz continuity.** Lipschitz continuity limits the model's sensitivity to input perturbations by restricting the magnitude of output variation. While this constraint improves robustness and stability, it does not impose any directional constraint on the input-output relationship (Khromov & Singh, 2024). A function can be Lipschitz continuous but violate monotonicity; conversely, a monotonic function may exhibit arbitrarily large local gradients. In practice, some monotonic neural network architectures implicitly control both properties, but the two constraints are inherently orthogonal (Nolte et al., 2023; Théate et al., 2023).
- **Monotonicity versus convexity.** Convexity imposes a global shape constraint on a function, restricting it to lie below all secant lines between points (Amos et al., 2017). In contrast, monotonicity only enforces directional consistency and does not impose curvature constraints. While a convex function may be monotonic along certain directions under appropriate conditions, a monotonic function is not necessarily convex or concave. This distinction is particularly important in neural networks because enforcing monotonicity alone is generally less restrictive than imposing convexity. Many studies have also begun to examine neural network architectures that explicitly combine monotonicity with convexity or concavity constraints, further highlighting the relatedness of these properties (Jadoon et al., 2026).
- **Monotonicity versus smoothness and differentiability.** Differentiability requires the existence of derivatives, whereas smoothness imposes stronger regularity conditions, typically involving continuity of derivatives. In contrast, monotonicity does not require smoothness; discontinuous or piecewise constant functions can also be monotonic. Conversely, a smooth function may not be monotonic. In neural networks, many monotonic architectures rely on smooth activation functions to improve optimization stability, but smoothness is neither a necessary nor a sufficient condition for monotonicity (Igel, 2024; Zhou et al., 2026).
- **Monotonicity and isotonic regression**. Isotonic regression provides a classic nonparametric estimator under monotonicity constraints, typically defined on partially ordered sets. While isotonic methods offer strong theoretical guarantees, they scale poorly with high-dimensional inputs and lack the representational power needed to capture complex feature interactions (Zhao et al., 2024a). Monotonic neural networks can be viewed as a flexible extension of isotonic ideas to high-dimensional and representation-learning settings, sacrificing exact optimality in favor of scalability and expressive power. This connection is also reflected in recent monotonic neural networks that explicitly draw inspiration from isotonic methods while improving smoothness, trainability, and applicability to complex data (You et al., 2017).

3. Taxonomy

In traditional machine learning, a wide range of models have been developed to incorporate monotonicity constraints, including decision trees (Khalaf et al., 2025; Potharst & Feelders, 2002; Yang et al., 2025), random forests (Bartley et al., 2019), lattice models and lookup tables (Bakst et al., 2021; Fiori et al., 2015; Milani Fard et al., 2016), boosting-based methods (Bartley et al., 2018; Leitenstorfer & Tutz, 2007), support vector machines (Chuang et al., 2024; Li & Chen, 2015), and classical statistical regression models (Kyng et al., 2015; Rodriguez et al., 2022). These approaches demonstrate that monotonicity has long been recognized as an important inductive bias for improving trustworthiness, interpretability, and consistency with domain knowledge. More recently, researchers have devoted increasing attention to incorporating monotonicity constraints into neural networks, motivated by their expressive power and widespread deployment in high-impact applications.

This section proposes a comprehensive taxonomy system for monotonicity constraints in neural networks. The taxonomy categorizes existing methods by how they implement monotonicity constraints. This categorization provides a unified perspective for analyzing the design trade-offs, guarantees, and limitations of different monotonicity constraints in neural networks.

3.1. Structure-based monotonicity approaches

Structure-based monotonicity approaches incorporate monotonicity into the model formulation through architectural components, constrained parameterizations, or hard feasibility constraints imposed during optimization, rather than relying solely on soft penalties or post-hoc verification. In compositional neural architectures, global monotonicity follows when the relevant components are coordinatewise monotone and all computational paths from the designated monotonic inputs to the output preserve the required direction. Other formulations impose ordering or derivative-sign constraints only under specific conditions or at selected points, resulting in conditional or local rather than global guarantees. Depending on the constrained input dimensions and the domain over which the constraints apply, these approaches can support total, partial, or per-dimension monotonicity. However, restrictive formulations may reduce expressive capacity. Conceptually, they follow the broader principle of shape-constrained modeling exemplified by isotonic regression (Kalbfleisch, 1975), monotone splines (Ramsay, 1988), and generalized additive models with shape constraints (Gomez-Rubio, 2018). Monotonic neural models extend this principle to neural function classes through mechanisms such as sign-constrained weights, monotone lattice parameters, positive-derivative parameterizations, and ordered basis-function weights. Despite this common objective, existing approaches differ substantially in their enforcement mechanisms, guarantee scope, expressive capacity, scalability, and support for different forms of monotonicity. The following subsections review these representative categories in detail.

3.1.1. Min-max monotonicity architectures

Min-max monotonicity architectures are among the earliest and most influential structure-based approaches for enforcing monotonicity in neural networks. Originally proposed by Sill (1997), these models represent monotonic functions as a minimum over groups of maximums of monotone affine functions. In this formulation, monotonicity is guaranteed by constraining the relevant affine weights to have the appropriate sign, while the nested max and min operators preserve monotonicity. This architecture provides an important early theoretical foundation for designing monotonic neural networks.

A min-max (MM) network is typically composed of multiple groups of linear units. Fig. 2 illustrates a four-layer min-max architecture with two hidden layers. Given an input vector $\mathbf{x} \in \mathbb{R}^d$, each linear unit j within group k computes an affine transformation of the form:

$$a^{(k,j)}(\mathbf{x}) = \mathbf{w}^{(k,j)} \cdot \mathbf{x} - b^{(k,j)}, \tag{6}$$

where $\mathbf{w}^{(k,j)} \in (\mathbb{R}_+)^d$ denotes a vector of non-negative weights and $b^{(k,j)} \in \mathbb{R}$ is a bias term. Within each group, the outputs of the linear units are aggregated using a maximum operator,

$$g^{(k)}(\mathbf{x}) = \max_{1 \le j \le h_k} a^{(k,j)}(\mathbf{x}), \tag{7}$$

where h_k denotes the number of units in group k. The final network output is then obtained by taking the minimum across all groups,

$$y(\mathbf{x}) = \min_{1 \le k \le K} g^{(k)}(\mathbf{x}), \tag{8}$$

yielding a piecewise-linear function that is monotonic with respect to all input dimensions subject to non-negativity constraints.

To achieve unconstrained optimization while preserving non-negative weights, researchers have proposed several parameterization strategies. Early work used exponential encodings (Sill, 1997) or squaring operations (Archer & Wang, 1993) to map unconstrained parameters

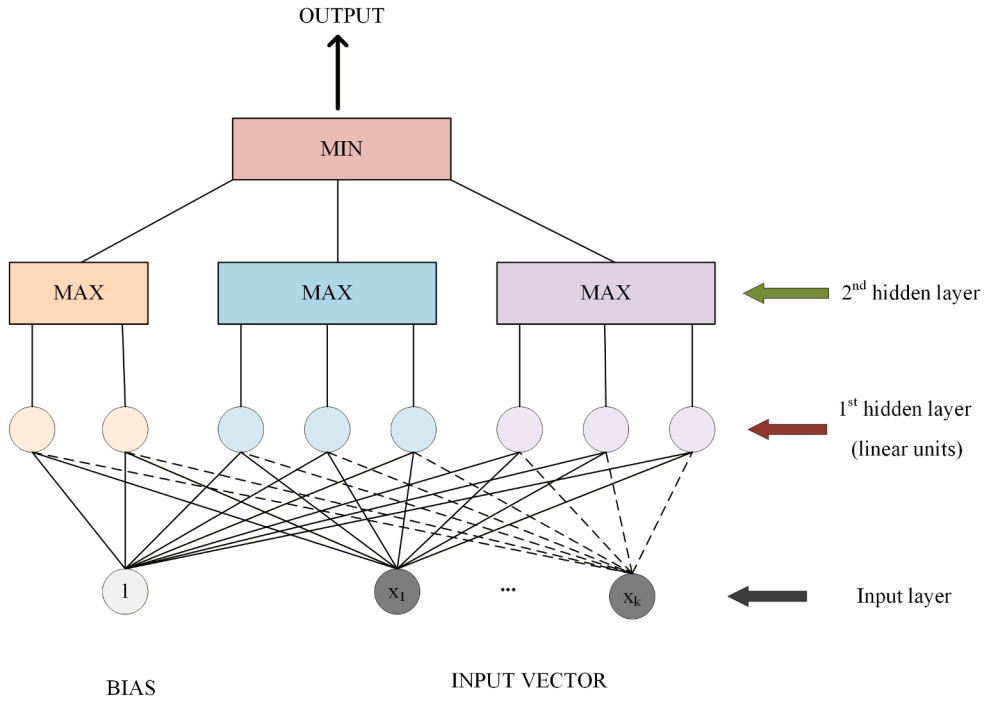


Fig. 2. Architecture of a four-layer neural network with min and max operators.

to the positive orthant. Later methods introduced smoothing transformations, such as the softplus function, to improve numerical stability during training (Dugas et al., 2009). Under the regularity conditions stated in their approximation theorem, Daniels and Velikova (2010) established that min-max networks can uniformly approximate continuous monotonic functions on compact domains. They further showed that the order of the minimization and maximization operators can be interchanged without loss of generality.

Smooth min-max (SMM) networks have been proposed to address optimization problems in the classic min-max architecture (Igel, 2024). MM modules can be sensitive to initialization and may contain inactive neurons that receive zero gradients on the observed inputs; moreover, the hard minimum and maximum operators are non-smooth. SMM addresses these problems by replacing the hard minimum and maximum with smooth LogSumExp counterparts that preserve monotonicity and retain MM networks' approximation properties.

For scalar arguments $z_1, \dots, z_m \in \mathbb{R}$ and $\beta > 0$, define

$$\begin{aligned} &\max_r z_r < \mathrm{LSE}_\beta(\mathbf{z}) \le \max_r z_r + \frac{\ln m}{\beta}, \\ &\min_r z_r - \frac{\ln m}{\beta} \le \mathrm{LSE}_{-\beta}(\mathbf{z}) < \min_r z_r. \end{aligned} \tag{9}$$

The SMM module is identical to the MM module except that the hard maximum within each group and the hard minimum across groups are replaced by

$$\begin{aligned} g^{(k)}_{\mathrm{SMM}}(\mathbf{x}) &= \mathrm{LSE}_\beta\left(a^{(k,1)}(\mathbf{x}), \dots, a^{(k,h_k)}(\mathbf{x})\right), \\ y_{\mathrm{SMM}}(\mathbf{x}) &= \mathrm{LSE}_{-\beta}\left(g^{(1)}_{\mathrm{SMM}}(\mathbf{x}), \dots, g^{(K)}_{\mathrm{SMM}}(\mathbf{x})\right). \end{aligned} \tag{10}$$

Because these operators are increasing in every argument, the affine sign constraints continue to imply monotonicity, while β controls the approximation–smoothness trade-off. SMM encodes β as a positive learnable parameter; the scaling can equivalently be absorbed into the affine parameters, leaving LSE_1 within groups and $\beta\,\mathrm{LSE}_{-1}$ at the output.

Min-max architectures face significant practical limitations. Their dependence on rigid structural combinations restricts expressive flexibility, complicates training dynamics, and limits compatibility with modern activation functions and deep architectures, thus limiting their application in large-scale and applied deep learning systems. Nevertheless, min-max architectures remain an important conceptual reference for studying monotonicity in neural networks. Many subsequent monotonic models can be interpreted as smooth relaxations, architectural generalizations, or functional analogs of the original min-max principle, highlighting their status as a cornerstone of monotonic function approximation theory.

3.1.2. *Lattice-based monotonicity architectures*

Lattice-based monotonicity architectures constitute a major paradigm for enforcing monotonicity in neural networks, combining flexible interpolation with explicit parameter constraints (Gupta et al., 2016; You et al., 2017). The core component of these methods is the lattice layer, which defines a multidimensional parametric grid with L^d vertices, where $L > 1$ is an integer hyperparameter controlling the grid's resolution in each input dimension, and d represents the dimension of the constrained input space (Gupta et al., 2016). Each vertex corresponds to a function value, and the predictions for any input are obtained through interpolation among adjacent vertices. This regular structure represents flexible monotonic functions while maintaining explicit control over shape constraints.

The Deep Lattice Network (DLN) framework, proposed by You et al. (2017), is a representative and widely adopted example of this paradigm. DLN combines multiple lattice layers in a similar ensemble architecture, often incorporating calibrators and linear embedding layers. Calibrators, one-dimensional piecewise linear functions, play a central role in DLN. They allow non-linear transformations of individual input features while maintaining monotonicity, enabling meaningful feature scaling before aggregation, and can serve as output layers to better align model predictions with target labels. Linear embedding layers further enhance expressiveness by mixing inputs before lattice interpolation while preserving the prescribed monotonic directions through non-negative weights on the designated monotonic inputs.

The DLN construction can be stated concretely. Each one-dimensional calibrator has a fixed input range $[a_{\min}, a_{\max}]$, K fixed uniformly spaced keypoints $a_1 < \cdots < a_K$, and trainable output parameters $\mathbf{b} \in [0,1]^K$. Its piecewise-linear interpolation is non-decreasing when

$b_k \le b_{k+1}$ for $k = 1, \dots, K-1$. For the multilinear interpolation used in the DLN experiments, an S-input lattice over $[0,1]^S$ has 2^S vertex parameters and evaluates

$$\ell(\mathbf{z}) = \sum_{\mathbf{v}\in\{0,1\}^S} \theta_{\mathbf{v}} \prod_{s=1}^{S} z_s^{v_s}(1-z_s)^{1-v_s}. \tag{11}$$

If coordinate j is designated as monotonic, monotonicity is enforced by constraining every pair of vertex parameters adjacent in that direction: $\theta_{\mathbf{v}+\mathbf{e}_j} \ge \theta_{\mathbf{v}}$ for every $\mathbf{v}$ satisfying $v_j = 0$. For a linear embedding, DLN separately embeds the monotonic and non-monotonic inputs and constrains every entry of the monotonic embedding matrix $\mathbf{W}^m$ to be non-negative, while the matrix associated with the non-monotonic inputs remains unconstrained. Outputs that depend monotonically on designated inputs are propagated as monotonic inputs to the following layer. Consequently, the composition of the appropriately constrained embedding, calibration, and lattice layers preserves end-to-end partial monotonicity.

Introducing monotonicity constraints into lattice models has been shown to produce a regularization effect. Empirical studies have shown that such constraints can improve generalization performance, especially when the distributions of training and test data are offset (Milani Fard et al., 2016). This property makes lattice-based models highly attractive in applications where robustness and consistency with domain knowledge are critical. Despite these advantages, an ordinary lattice with k vertices per dimension requires k^d vertex parameters. The Hierarchical Lattice Layer (HLL) (Yanagisawa et al., 2022) partitions $\mathbf{x} = (\mathbf{p}, \mathbf{q})$, where $\mathbf{p}$ contains the unconstrained features and $\mathbf{q} \in [0,1]^m$ contains the designated monotonic features, and constructs only the m-dimensional vertex grid of size k^m. A fixed total order specifies the recursive evaluation order. For each vertex, already evaluated dominated and dominating vertices determine valid lower and upper values, and an auxiliary network outputs a coefficient in $[0,1]$ that places the new vertex value between these bounds. Multilinear or simplex interpolation then extends the ordered vertex values to arbitrary $\mathbf{q}$. This parameterization guarantees partial monotonicity while allowing standard gradient-based training. It transfers the exponential grid dependence from the full dimension d to the number of monotonic features m; the remaining k^m factor can nevertheless be prohibitive when m is large. Beyond monotonicity, lattice models can encode additional shape constraints, including diminishing or accelerating returns (Cotter et al., 2019; Gupta et al., 2018).

Despite the success of explicit lattice parameterization, researchers have recently explored other order-preserving representations that relax constraints on discrete grid construction while maintaining the architecture's monotonicity. These models go beyond vertex-based interpolation, enforcing monotonic behavior through embedding mechanisms or function decomposition. These advances reflect a broader trend toward the development of more expressive and scalable monotonicity architectures, which will be discussed in Section 3.1.5.

Overall, lattice-based approaches provide a powerful and theoretically sound tool for modeling monotonic relationships. However, their computational complexity requires careful architecture design, especially in high-dimensional environments, where more conventional neural network architectures may scale better.

3.1.3. Monotonic extensions of classical architectures

Alongside specially designed monotonic architectures, a complementary line of research extends classic neural network models to incorporate monotonicity constraints. The core motivation behind these methods is to retain the flexibility, simplicity, and ease of understanding of existing architectures while enabling them to adhere to monotonic relationships prescribed by domain knowledge. Typically, these extensions introduce monotonicity by modifying the objective function, structural constraints, or training process, rather than redesigning the network architecture from scratch.

A fundamental mechanism behind many monotonic extensions of classical neural network architectures is combining sign-constrained weights with monotonic activation functions. In feedforward networks, constraining selected weights to be non-negative and using non-decreasing activation functions ensures that the resulting network is monotonic with respect to the designated inputs (Archer & Wang, 1993). More precisely, Daniels and Velikova (2010) prove that, for any continuous coordinatewise non-decreasing function $f : K \to \mathbb{R}$ on a compact set $K \subset \mathbb{R}^k$ and any $\varepsilon > 0$, there exists a positive-weight feedforward network with sigmoid activations and at most k hidden layers whose output O satisfies $|O(\mathbf{x}) - f(\mathbf{x})| < \varepsilon$ for every $\mathbf{x} \in K$. The result is therefore depth- and activation-dependent, rather than a consequence of width alone. This simple yet influential principle underpins a wide range of early monotonic neural models and motivates many later extensions of classical architectures.

The Monotonic Classification Extreme Learning Machine (MCELM) (Zhu et al., 2017) is a typical example, extending the Extreme Learning Machine (ELM) framework (Huang et al., 2006) to monotonic classification tasks. ELM is characterized by random initialization of hidden-layer parameters and analytical determination of output weights, thus enabling rapid training. MCELM directly incorporates monotonicity constraints into the learning objective by modeling the training process as a quadratic programming problem. Specifically, the optimization objective minimizes the difference between the predicted output and the true label while forcing the partial derivatives of the output function with respect to selected input features to satisfy non-negativity (or non-positivity) conditions, according to the desired monotonicity. When the derivative-sign constraints are feasible and satisfied under the stated sigmoid hidden-unit model, they provide the required monotonicity; the constrained feasible set may nevertheless reduce flexibility relative to an unconstrained ELM.

Radial basis function (RBF) networks (Husek, 2018, 2024; Lowe & Broomhead, 1988) provide another family of monotonic extensions of classical shallow models. URBF constrains derivative signs at finitely many monotonicity points and therefore provides only a sampled-domain guarantee. NRBF instead normalizes Gaussian basis responses and can provide a sufficient global guarantee, but only under the shared-kernel, center-placement, and weight-ordering assumptions stated below.

These shallow extensions also illustrate the distinction between pointwise and global constraints. Let $\mathbf{x} = (x_1, \dots, x_d)^\top \in \mathbb{R}^d$ be the ELM input and let H be the number of hidden nodes. For hidden node r, $\mathbf{w}_r = (w_{r1}, \dots, w_{rd})^\top$ and b_r denote its randomly generated input-weight vector and bias, respectively, while β_r denotes its output weight. With the sigmoid activation σ and $\sigma_r = \sigma(\mathbf{w}_r^\top \mathbf{x} + b_r)$, the ELM output is $o(\mathbf{x}) = \sum_{r=1}^{H} \beta_r \sigma_r$, and its derivative with respect to feature x_j, $j \in \{1, \dots, d\}$, is

$$\frac{\partial o}{\partial x_j} = \sum_{r=1}^{H} \beta_r w_{rj} \sigma_r (1-\sigma_r). \tag{12}$$

Because $\sigma_r(1-\sigma_r) > 0$, MCELM uses the sufficient per-hidden-unit constraints $\beta_r w_{rj} > 0$ for an increasing feature and $\beta_r w_{rj} < 0$ for a decreasing feature, for $r = 1, \dots, H$. With the randomly generated input weights and biases fixed, these constraints are linear in the output-weight vector $\boldsymbol{\beta} = (\beta_1, \dots, \beta_H)^\top$, which is estimated through the quadratic program

$$\min_{\boldsymbol{\beta}} \frac{1}{2}\boldsymbol{\beta}^\top \mathbf{C}\boldsymbol{\beta} + \boldsymbol{\eta}^\top \boldsymbol{\beta}, \qquad \mathbf{A}\boldsymbol{\beta} \le \mathbf{B}, \tag{13}$$

where $\mathbf{C} \in \mathbb{R}^{H\times H}$ is the symmetric Hessian matrix of the squared-error objective, $\boldsymbol{\eta} \in \mathbb{R}^H$ contains its linear coefficients, $\mathbf{A}$ contains the coefficients of the derivative-sign constraints, and $\mathbf{B}$ is the corresponding zero vector in the MCELM formulation. These sufficient constraints make the required derivative sign hold throughout the input domain (Zhu et al., 2017). For URBF, imposing analogous derivative inequalities only at a finite set of monotonicity points $\mathcal{Z} \subset \mathcal{X}$ yields linear constraints on the RBF weights but only a sampled-domain guarantee (Husek, 2018).

The exact NRBF result requires a more specific construction. Let M be the number of units, α_r the output weight, and $\mathbf{c}_r \in \mathbb{R}^d$ the center of unit r. All units use Gaussian kernels with the same positive-definite diagonal matrix

$$\phi_r(\mathbf{x}) = \exp\left[-(\mathbf{x}-\mathbf{c}_r)^\top \mathbf{Q}(\mathbf{x}-\mathbf{c}_r)\right], \qquad \mathbf{Q} = \operatorname{diag}\left(\frac{1}{2\sigma_1^2}, \dots, \frac{1}{2\sigma_d^2}\right) \succ 0, \tag{14}$$

and the normalized output is

$$f(\mathbf{x}) = \frac{\sum_{r=1}^{M} \alpha_r \phi_r(\mathbf{x})}{\sum_{r=1}^{M} \phi_r(\mathbf{x})}. \tag{15}$$

Under these equal principal-axis-parallel Gaussian kernels, a sufficient condition for global non-decreasing dependence on coordinate x_j is $\alpha_p \le \alpha_q$ whenever $c_{p,j} < c_{q,j}$; the inequalities are reversed for a non-increasing dependence. When monotonicity is imposed in several coordinates, the centers lie on a rectangular grid so compatible weight orderings can be enforced along the corresponding grid directions. These conditions do not extend automatically to arbitrary radial basis functions. Thus, URBF supplies only a sampled-domain constraint, whereas NRBF supplies a global guarantee only under the stated Gaussian-kernel, shared-width, center-layout, and weight-ordering assumptions (Husek, 2024). Center placement and the shared widths affect both approximation quality and the feasibility of these sufficient constraints.

Beyond these constrained variants of classical shallow architectures, later work extended the same design philosophy to more flexible deep neural models. A representative example is the dual-stream architecture proposed by Silva et al. (2018), which explicitly separates monotonic and non-monotonic features. In this design, monotonic features are processed by a sub-network with sign-constrained weights, while non-monotonic features are handled by an unconstrained sub-network. The outputs are then merged and passed through subsequent positively constrained layers, allowing the network to preserve monotonicity with respect to selected inputs while retaining flexibility for unconstrained features.

Following a related idea, MonoNet (Nguyen et al., 2023) enhances interpretability by separating an unconstrained representation-learning block from a monotonic prediction block. Consider a multilayer perceptron whose layers are indexed by $0, \dots, L$, with input $\mathbf{h}^{(0)} = \mathbf{x}$ and output $\mathbf{h}^{(L)} = \mathbf{y}$, and let $\mathbf{h}^{(k)}$ be one chosen interpretable hidden layer. The layers before $\mathbf{h}^{(k)}$ remain unrestricted and may learn arbitrary high-level features. Let $\widetilde{\mathbf{h}}^{(k)}$ and $\widetilde{\mathbf{y}}$ denote, respectively, the rescaled input and output of the subsequent monotonic block. For example, a layer within this block can use the positive-weight parameterization

$$\mathbf{z}^{(\ell+1)} = \sigma\left(\exp\left(\mathbf{W}^{(\ell)}\right)\mathbf{z}^{(\ell)} + \mathbf{b}^{(\ell)}\right), \qquad \ell = k, \dots, L-1, \tag{16}$$

where $\mathbf{z}^{(k)} = \widetilde{\mathbf{h}}^{(k)}$, $\mathbf{z}^{(L)} = \widetilde{\mathbf{y}}$, the exponential is applied elementwise, and σ is a non-decreasing activation function. Hence every entry of the effective weight matrix is positive, and stacking such layers makes $\widetilde{\mathbf{y}}$ coordinatewise non-decreasing with respect to $\widetilde{\mathbf{h}}^{(k)}$. The elementwise exponential is one possible positive-valued transformation; the original construction also notes that other such transformations may be used. To permit increasing or decreasing relationships, MonoNet introduces component-wise rescaling at the boundaries of the monotonic block,

$$\widetilde{\mathbf{h}}^{(k)} = \boldsymbol{\alpha} \odot \mathbf{h}^{(k)}, \qquad \mathbf{y} = \boldsymbol{\beta} \odot \widetilde{\mathbf{y}}, \tag{17}$$

where $\boldsymbol{\alpha} \in \mathbb{R}^{N^{(k)}}$, $\boldsymbol{\beta} \in \mathbb{R}^{N^{(L)}}$, and $\odot$ denotes component-wise multiplication. The sign of $\alpha_i \beta_j$ determines whether output component y_j is non-decreasing or non-increasing with respect to interpretable feature $h_i^{(k)}$; a zero product gives a constant, and thus non-strictly monotone, relationship. Importantly, this exact guarantee concerns the mapping from the selected hidden representation $\mathbf{h}^{(k)}$ to $\mathbf{y}$; because the mapping from the original input $\mathbf{x}$ to $\mathbf{h}^{(k)}$ is unconstrained, MonoNet does not by itself guarantee monotonicity with respect to the original input variables.

Compared with earlier constrained shallow models, these architectures show how classical monotonic design principles can extend to modern deep learning settings, improving interpretability and representation flexibility. A related recent direction extends isotonic regression itself into a differentiable neural module. Cheng et al. (2026) propose the isotonic layer, a plug-and-play architectural component that integrates piecewise linear isotonic fitting into end-to-end deep models. By enforcing non-negative segment-wise slopes and learning context-dependent isotonic embeddings, the method guarantees global monotonicity while supporting fine-grained calibration and debiasing in recommendation systems. Unlike full end-to-end monotonic predictors, this line of work highlights an emerging modular view of monotonicity, in which monotonic layers are embedded inside larger neural systems as reusable calibration or debiasing components.

In summary, these extensions illustrate an important transitional phase in monotonic learning, in which classical models are modified by monotonic constraints, but are increasingly being replaced by architectures that integrate monotonicity more deeply into modern deep learning frameworks.

3.1.4. Flow-based monotonicity architectures

Flow-based architectures construct invertible multivariate maps from scalar transformations that are strictly monotonic in the currently transformed coordinate. In autoregressive flows, conditioning on preceding coordinates yields a triangular map, so strict scalar monotonicity ensures invertibility and makes the Jacobian determinant tractable.

Normalizing flows (NFs) (Ghugare & Eysenbach, 2025; Gu et al., 2025; Kobyzev et al., 2021) constitute a powerful class of generative models that learn invertible transformations between simple base distributions and complex target distributions. Although developed primarily for density estimation, variational inference, and generative modeling, they also provide an important architectural setting in which monotonicity guarantees invertibility. Classical autoregressive flows, such as masked autoregressive flows (MAFs) (Liu et al., 2024; Papamakarios et al., 2017; Scheidt et al., 2025) and inverse autoregressive flows (IAFs) (Kingma et al., 2016; Levy et al., 2023; Majerz et al., 2025), use conditional affine scalar transformations. Neural autoregressive flows (NAFs) (Huang et al., 2018; Wei & Xi, 2025) replace the affine transformer with a neural transformer. After fixing an ordering of the input coordinates, the tth component of the autoregressive transformation is written as

$$y_t = f_t(\mathbf{x}_{1:t}) = \tau_{\boldsymbol{\phi}_t}(x_t), \qquad \boldsymbol{\phi}_t = c(\mathbf{x}_{1:t-1}), \tag{18}$$

where c is the autoregressive conditioner and $\tau_{\boldsymbol{\phi}_t}$ is a scalar neural transformer whose pseudo-parameters are supplied by c. Proposition 1 of Huang et al. (2018) states that strictly positive weights and strictly increasing activation functions are sufficient to make the transformer strictly increasing and hence invertible. The deep sigmoidal flow (DSF) and deep dense sigmoidal flow (DDSF) transformers satisfy this condition, so $\partial y_t / \partial x_t > 0$. Because y_t depends only on $\mathbf{x}_{1:t}$, the Jacobian of the complete forward transformation is lower triangular, giving

$$\log\left|\det \boldsymbol{J}_f(\mathbf{x})\right| = \sum_{t=1}^{d} \log\left(\frac{\partial y_t}{\partial x_t}\right). \tag{19}$$

Compared with an affine transformer, DSF and DDSF can introduce inflection points and thereby represent more complex conditional densities. For DSF, the universal density-approximation result assumes positive, continuous source and target densities, with the source supported on an open set.

The unconstrained monotonic neural network (UMNN) (Théate et al., 2023; Wehenkel & Louppe, 2019) takes a different route: instead of imposing sign constraints throughout the scalar transformer, it models a strictly positive derivative with a free-form neural network. Specifically,

$$F(x;\psi) = \beta + \int_0^x f(t;\psi)\,\mathrm{d}t, \qquad \beta = F(0;\psi), \quad f(t;\psi) > 0. \tag{20}$$

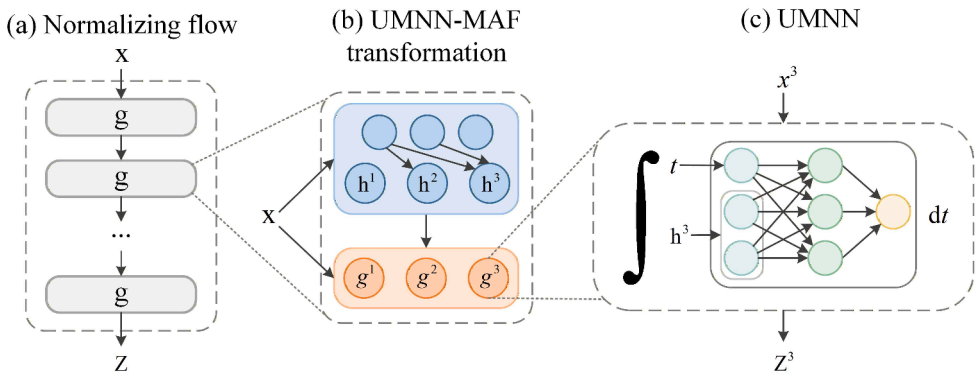


Fig. 3. (a) A normalizing flow made of repeated UMNN-MAF transformations g with identical architectures. (b) A UMNN-MAF which transforms a vector $x \in \mathbb{R}^3$. (c) The UMNN network used to map x_3 to z_3 conditioned on the embedding $h^3(x_{1:2})$.

Since $f(t;\psi) > 0$, the exact integral defines a strictly increasing and hence invertible transformation. In the original implementation, the otherwise unconstrained integrand network enforces positivity through an ELU-plus-one output, and the integral is approximated using static Clenshaw–Curtis quadrature. Thus, quadrature resolution and integrand smoothness affect numerical approximation accuracy, whereas strict positivity establishes the monotonicity of the exact integral transformation. The integration error can be reduced by controlling the integrand's Lipschitz constant and selecting a suitable number of quadrature points (Gouk et al., 2021). A general UMNN has no closed-form inverse, but strict monotonicity ensures a unique inverse that can be computed using root-finding methods such as bisection (Wehenkel & Louppe, 2019). When incorporated into an autoregressive structure, UMNN-MAF retains a triangular Jacobian and extends the scalar construction to multivariate density estimation. Fig. 3 illustrates the UMNN-MAF framework and its underlying integral transformation. Beyond density estimation, UMNN-based transformations have also been applied in ranking ensembles and recommender systems to perform monotonic score calibration and fusion while maintaining ranking consistency (Bai et al., 2025; Xu et al., 2025).

Flow-based monotonicity architectures were developed primarily for density estimation and generative modeling, where monotonic scalar transformations ensure invertibility rather than encode domain-driven feature–prediction relations. They are included because they provide an expressive architectural route to monotonicity, although they may be less natural for supervised tasks centered on interpretable, feature-specific constraints.

3.1.5. *Expressive and scalable monotonicity architectures*

The methods in this group pursue greater expressive flexibility or improved computational scaling while retaining structural monotonicity guarantees. These are design objectives rather than properties established uniformly across all methods: their guarantees, architectural costs, and high-dimensional behavior remain method-dependent.

A central motivation is the expressive limitation of positive-weight networks with convex activations such as ReLU, which are restricted to convex monotonic functions (Gagneux et al., 2025; Mikulincer & Reichman, 2025; Nair & Hinton, 2010; Rössig & Petkovic, 2021). Sartor et al. (2025) refine this picture with a constant-depth construction: an MLP with non-negative weights and three hidden layers can interpolate any finite sample from a monotone non-decreasing function when the monotone activations alternate their saturation side. They also give a separate four-layer construction with non-positive weights that yields universal approximation for monotone functions when the monotone activation saturates on at least one side; this class includes convex one-sided-saturating activations such as ReLU. These results show that expressiveness depends on the interaction between sign constraints, depth, and activation saturation rather than on weight signs alone.

The Lipschitz Monotonic Network (LMN) (Nolte et al., 2023) starts from an unrestricted-sign scalar network g whose Lipschitz constant with respect to the ℓ_1 input norm is bounded by λ, and adds a single end-to-end linear residual for the set S of inputs required to be non-decreasing:

$$\left|g(\mathbf{x}) - g(\mathbf{x}')\right| \le \lambda \|\mathbf{x} - \mathbf{x}'\|_1, \qquad f(\mathbf{x}) = g(\mathbf{x}) + \lambda \sum_{i \in S} x_i. \tag{21}$$

For any $i \in S$ and $\delta \ge 0$, let $\mathbf{e}_i$ denote the ith standard basis vector. The Lipschitz bound gives

$$f(\mathbf{x} + \delta \mathbf{e}_i) - f(\mathbf{x}) \ge -\lambda\delta + \lambda\delta = 0, \tag{22}$$

so LMN is globally non-decreasing in the features in S. No monotonicity direction is imposed on features outside S, although they remain subject to the Lipschitz constraint on g. A decreasing direction can be handled by reversing the sign of the corresponding input. For a base network with D weight layers, a sufficient construction bounds the product of the induced operator norms associated with ℓ_1, for example $\prod_{\ell=1}^{D} \|W^{(\ell)}\|_1 \le \lambda$, and uses activations with Lipschitz constants at most one. Under the layer-wise induced-ℓ_∞ norm constraints studied by Huster et al. (2018), unit-norm-constrained ReLU networks exhibit severe expressive limitations and cannot approximate even $x \mapsto |x|$ arbitrarily well. To retain expressive power under Lipschitz control, LMN uses gradient-norm-preserving 1-Lipschitz activations such as GroupSort (Anil et al., 2019). Under the mixed-norm parameterization, GroupSort networks with group size two are universal approximators of scalar 1-Lipschitz functions on compact domains. Subsequent work provides explicit depth and size bounds for approximating Lipschitz functions under related GroupSort norm constraints (Tanielian & Biau, 2021). These results do not establish universality for LMN's alternative product-of-induced-ℓ_1-norm layer-wise construction.

Constrained monotonic neural networks (CoMNN) (Runje & Shankaranarayana, 2023) construct a monotonic dense unit by assigning different neurons, through a fixed activation-selection vector, to a base convex activation, its point-reflected concave counterpart, or a bounded saturated activation. This activation pairing is related to constructions such as concatenated ReLU (Banerjee et al., 2019; Shang et al., 2016), but it is used here to represent both convex and concave segments without losing the prescribed derivative directions. Fig. 4 illustrates the resulting constrained fully connected building block.

For an input $\mathbf{x} \in \mathbb{R}^k$, let $t_i \in \{-1, 0, 1\}$ encode decreasing, unconstrained, and increasing dependence on x_i, and let W_{ji} denote the weight from input i to unit j. CoMNN transforms each input column according to

$$\widetilde{W}_{ji} = \begin{cases} |W_{ji}|, & t_i = 1, \\ -|W_{ji}|, & t_i = -1, \\ W_{ji}, & t_i = 0, \end{cases} \tag{23}$$

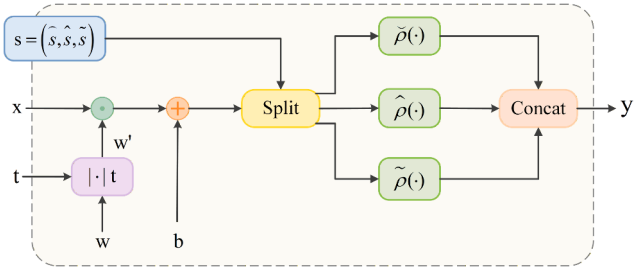


Fig. 4. Proposed monotonic dense unit or constrained monotone fully connected layer.

and applies the selected non-decreasing activation to $\widetilde{W}\mathbf{x}+\mathbf{b}$. This column-wise transformation is applied in the input layer, whereas subsequent constrained dense layers use an all-one indicator vector, so their non-decreasing activations preserve the derivative directions established by the first layer. Inputs with $t_i = 0$ remain unrestricted. The construction assumes a zero-centered, non-decreasing, convex, and lower-bounded base activation; its point reflection and bounded branch are then derived from this base rather than requiring every neuron to use a convex activation. In particular, the saturated-activation construction is essential to the universal-approximation result: any continuous monotone function on a compact subset of $\mathbb{R}^k$ can be approximated arbitrarily well by the resulting constrained network with at most k layers.

Scalable monotonic neural networks (SMNN) (Kim & Lee, 2024) use a prescribed partially connected architecture built from exponentiated, ReLU, and confluence units. In the first hidden layer, the exponentiated unit receives the monotonic features, whereas the ReLU and confluence units receive the non-monotonic features. From the second layer onward, the preceding exponentiated and confluence outputs are concatenated and passed to the next exponentiated unit; the unrestricted ReLU stream supplies the confluence outputs that enter this later monotonic path. Consequently, contextual information from non-monotonic variables can influence the prediction without creating a negative-weight path from a constrained input. This fixed block can be stacked and trained end-to-end by standard backpropagation. Fig. 5 illustrates this information flow.

For a monotonic feature vector $\mathbf{x}_m$, a first-layer exponentiated unit has the explicit form

$$h_{\exp}(\mathbf{x}_m) = \text{ReLU-}n\big(\exp(\mathbf{w})^\top \mathbf{x}_m + b\big), \qquad \text{ReLU-}n(u) = \min\{n, \max\{0, u\}\}. \tag{24}$$

Here, $n > 0$ is the saturation threshold. For a later layer $\ell \geq 2$, the corresponding input is the concatenation $[h_{\exp}^{(\ell-1)}; h_{\text{conf}}^{(\ell-1)}]$, and the output layer also uses exponentiated weights. Because the exponential is positive and ReLU-n is non-decreasing, the chain rule gives $\partial f/\partial x_j \geq 0$ for every designated non-decreasing feature; a non-increasing constraint is implemented by negating that input (Kim & Lee, 2024). The bounded ReLU-n activation (Liew et al., 2016) is motivated by the universal-approximation result for sufficiently deep positive-weight networks with threshold activations; SMNN does not claim a separate universal-approximation theorem for its complete partially connected architecture.

The Deep Isotonic Embedding Network (DIEN) (Zhao et al., 2024a) separates monotonic and non-monotonic inputs among four modules. Isotonic embedding modules transform each designated monotonic scalar into an order-preserving vector, and a monotonic feature learning network with non-negative weights models interactions among the monotonic variables. A bias learning network maps the non-monotonic variables to an additive bias, while a multi-weight learning network uses the same variables to produce context-dependent non-negative weights for the embeddings. Fig. 6 illustrates these modules and their final combination.

In DIEN, a monotonic scalar x_i^m is compared with ordered reference values $d_1 < \cdots < d_D$ to form an embedding $\mathbf{e}_i(x_i^m)$ consisting of leading ones, at most one linearly interpolated component, and trailing zeros. Hence $x_i^m \leq x_i^{m\prime}$ implies $\mathbf{e}_i(x_i^m) \leq \mathbf{e}_i(x_i^{m\prime})$ componentwise. The non-monotonic stream produces a context-dependent but non-negative vector $\mathbf{w}_i(\mathbf{x}^{nm})$, and the corresponding contribution is

$$\ell_i(\mathbf{x}) = \mathbf{w}_i(\mathbf{x}^{nm})^\top \mathbf{e}_i(x_i^m). \tag{25}$$

Holding $\mathbf{x}^{nm}$ fixed, this inner product is non-decreasing in x_i^m. The monotonic feature learning network uses non-negative weights together with non-decreasing ReLU or Softplus activations, so its output $m(\mathbf{x}^m)$ is also componentwise non-decreasing in the designated monotonic features. If $b(\mathbf{x}^{nm})$ denotes the bias-learning output, DIEN first forms

$$s(\mathbf{x}) = \sum_i \ell_i(\mathbf{x}) + m(\mathbf{x}^m) + b(\mathbf{x}^{nm}). \tag{26}$$

For regression, the prediction is this linear sum; for binary classification, DIEN applies a monotonically increasing sigmoid to $s(\mathbf{x})$. The sigmoid therefore preserves the exact global ceteris-paribus monotonicity of s with respect to the designated monotonic features.

In a distinct spline-based direction, MonoKAN (Polo-Molina et al., 2026b) adapts Kolmogorov–Arnold Networks (KANs) (Liu et al., 2025), whose layers aggregate learnable univariate functions placed on the edges, to prediction with certified partial monotonicity. For layer ℓ, its computation can be written as

$$x_{\ell+1,j} = \sum_{i=1}^{n_\ell} \left[\omega^{\phi}_{\ell,j,i}\phi_{\ell,j,i}(x_{\ell,i}) + \omega^{\rho}_{\ell,j,i}\rho(x_{\ell,i})\right] + \theta_{\ell,j}, \tag{27}$$

where $x_{\ell,i}$ is the activation of node i in layer ℓ, n_ℓ is the number of nodes in that layer, $\phi_{\ell,j,i}$ is a learnable edge spline, ρ is a non-decreasing base activation, $\omega^{\phi}_{\ell,j,i}$ and $\omega^{\rho}_{\ell,j,i}$ are their corresponding weights, and $\theta_{\ell,j}$ is a bias. MonoKAN replaces the B-splines of the original KAN with cubic Hermite splines and imposes sufficient monotonicity conditions on their knot values and slopes, together with direction-consistent sign constraints on the relevant weights. Linear extrapolation using the boundary slopes extends the guarantee beyond the knot interval, yielding certified partial monotonicity with respect to the designated inputs over the entire input space while retaining the ability to represent monotone non-convex functions. After each optimizer update, clamping projects the constrained parameters back onto the feasible set. Although this projection introduces little reported overhead relative to the underlying Hermite-KAN, the use of a separate spline on each inter-layer edge can still limit scalability in high-dimensional settings.

In summary, these expressive and scalable architectures combine monotonicity with residual connections, Lipschitz constraints, specialized activations, embeddings, or spline decompositions. They expand the available design space, but neither expressive efficiency nor high-dimensional scalability follows uniformly: architectural complexity, parameter growth, and guarantee scope remain method-dependent.

3.1.6. *Summary of structure-based monotonicity approaches*

Structure-based monotonicity approaches enforce ordering relations through architectural design or hard parameter constraints, thereby providing direct monotonicity guarantees without relying solely on soft training penalties or post-hoc verification. Despite substantial differences in formulation, these methods share the principle of embedding monotonicity into the model construction itself. However, the resulting guarantees vary from exact global monotonicity to local or conditional monotonicity, depending on how and where the constraints are imposed.

The reviewed approaches differ in mechanism, expressive power, scalability, and guarantee scope. In Table 1, “Monotonic scope” identifies where monotonicity is imposed: “Total” covers all input coordinates, “Partial” covers selected input coordinates, and “Per-dimension” denotes a scalar transformation within a multivariate map. “Guarantee”

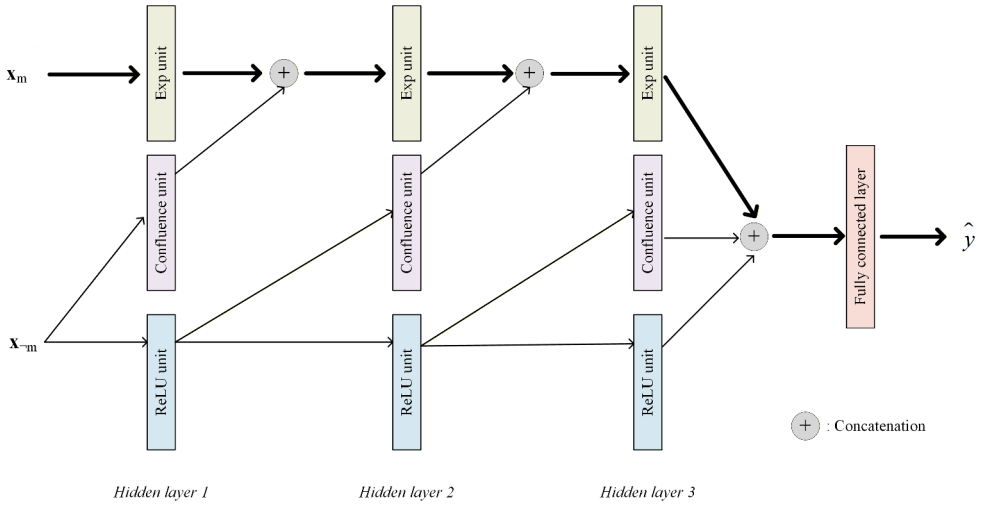


Fig. 5. SMNN structure. The features are divided into two groups based on whether each variable requires monotonicity (x_m) or not ($x_{\neg m}$). The monotonic features are directed exclusively through the exponentiated unit and fully connected layer to ensure monotonicity, as indicated by the bold line.

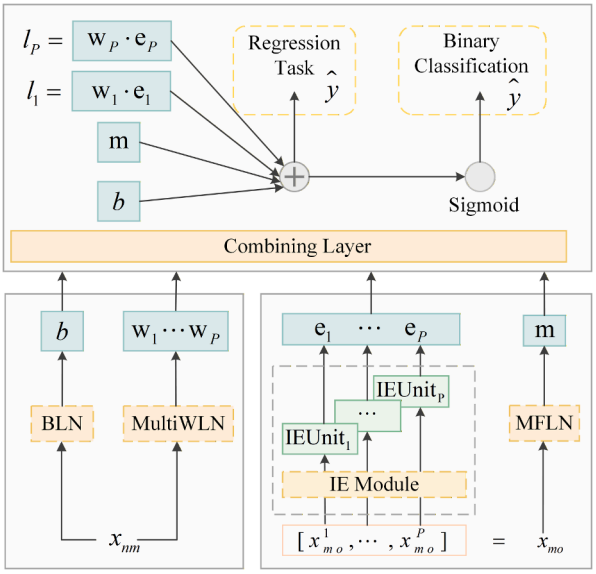


Fig. 6. Overall structure of DIEN.

describes its strength and domain: “Exact” is global on the stated domain, “Local” applies only at selected points, and “Conditional” holds only under the stated assumptions and feasibility conditions.

3.2. Non-structural monotonicity approaches

Unlike structure-based approaches, non-structural monotonicity approaches promote, assess, or certify monotonic behavior primarily through training objectives, optimization procedures, formal verification, or prediction-time correction rather than an end-to-end monotone parameterization. Many of these approaches are compatible with standard neural architectures, although some hybrid formulations incorporate specialized components and some verification frameworks require formally encodable network classes. This flexibility makes them attractive in practice, while their guarantee strength, applicability, and computational cost depend on the particular enforcement or verification mechanism. The following subsections review these representative categories in detail.

3.2.1. Training-based monotonicity induction mechanisms

Training-based monotonicity induction mechanisms augment the learning objective with soft constraints, penalty terms, or constraint-aware sampling strategies. Many can be applied to standard differentiable networks, although some hybrid formulations also prescribe how monotonic and unconstrained features are processed. Because the relevant constraints are generally evaluated at finitely many sampled points, these methods typically provide empirical monotonicity rather than an unconditional global guarantee.

Two key design dimensions characterize training-based monotonicity induction mechanisms. The first concerns how to select the constraint points used to evaluate monotonicity violations. Different strategies, including empirical data sampling, uniform sampling of the input space, and interpolation-based schemes, lead to substantially different trade-offs among effectiveness, robustness, and computational cost. The second concerns how monotonicity violations themselves are defined. While many methods penalize undesirable gradient signs or reversed local comparisons, more recent approaches introduce explicit reference trends against which they evaluate monotonic behavior.

Early research in this direction focused on incorporating explicit prior knowledge about the target function into learning. Abu-Mostafa (1990), Sill and Abu-Mostafa (1996) introduced the concept of monotonicity hints (MHs), namely auxiliary information that goes beyond standard input-output examples. These hints indicate that the target function should be monotonic with respect to at least some of the input variables. The authors proposed a monotonicity error to quantify violations of this prior knowledge and derived a Bayesian-inspired objective function that balances empirical data fitting with monotonicity penalties. This formulation established an early and influential framework for inducing soft monotonicity in neural network training.

Muralidhar et al. (2018) propose Domain-Adapted Neural Networks (DANN), which incorporate supplied domain rules through an additional loss term:

$$\min_{\theta} \ \mathcal{L}_{\text{data}}(\theta) + \lambda_D \mathcal{L}_D(\theta) + \lambda_R \mathcal{R}(\theta), \tag{28}$$

where $\mathcal{L}_{\text{data}}$ is the predictive data-fitting loss, $\mathcal{L}_D$ measures violations of the supplied domain rules, $\mathcal{R}(\theta)$ is an L_2 regularization term controlling model complexity, and λ_D and λ_R are non-negative hyperparameters weighting the domain loss and the regularizer, respectively. For a desired monotonic ordering, a Boolean mask selects pairs for which the supplied input ordering holds but the predicted ordering is reversed, and a ReLU term penalizes the magnitude of that reversal. DANN therefore encourages the specified rules on generated or observed pairs, but it

Table 1
Representative summary of structure-based monotonicity approaches.

Method	Abbreviation	Core mechanism	Monotonic scope	Guarantee	Reference	Code
Min-max architectures						
Min-max network	MM	Max over affine units within groups followed by min aggregation; non-negative weights enforce monotonicity.	Total	Exact	(Sill, 1997)	–
Smooth min-max network	SMM	Replace hard min-max with smooth LogSumExp operators for stable optimization.	Total / Partial	Exact	(Igel, 2024)	https://github.com/christian-igel/SMM
Lattice-based architectures						
Deep lattice network	DLN	Composition of calibrators, lattice layers, and linear embedding layers for partially monotone functions.	Partial / Total	Exact	(You et al., 2017)	https://github.com/tensorflow/lattice
Hierarchical lattice layer	HLL	Hierarchical lattice over designated monotonic features, with complexity exponential in their number.	Partial	Exact	(Yanagisawa et al., 2022)	https://ibm.github.io/pmlayer
Extensions of classical architectures						
Monotonic classification extreme learning machine	MCELM	Quadratic programming with derivative sign constraints in Extreme Learning Machines.	Partial	Conditional	(Zhu et al., 2017)	–
Unnormalized radial basis function	URBF	Enforce derivative sign constraints at selected monotonicity points.	Partial	Local	(Husek, 2018)	–
Normalized radial basis function	NRBF	Order weights of shared-width, axis-parallel Gaussian kernels; multiple constrained directions use grid-aligned centers.	Partial / Total	Exact	(Husek, 2024)	–
MonoNet	MonoNet	Separates unrestricted representation learning from a monotonic prediction module with sign-constrained weights and monotonic activations.	Partial	Exact	(Nguyen et al., 2023)	https://github.com/phineasng/mononet
Flow-based architectures						
Neural autoregressive flow	NAF	Autoregressive monotone transformations ensuring invertibility.	Per-dimension	Exact	(Huang et al., 2018)	https://github.com/CW-Huang/NAF
Unconstrained monotonic neural network	UMNN	Parameterize strictly positive derivatives and integrate to obtain monotone functions.	Per-dimension	Exact	(Wehenkel & Louppe, 2019)	https://github.com/AWehenkel/UMNN
Expressive and scalable architectures						
Lipschitz monotonic network	LMN	ℓ_1-Lipschitz base network plus a directionally signed linear residual.	Partial	Exact	(Nolte et al., 2023)	https://github.com/niklasnolte/MonotonicNetworks
Constrained monotonic neural network	CoMNN	Convex, point-reflected concave, and bounded saturated activation branches under direction-aware weight signs.	Partial	Exact	(Runje & Shankaranarayana, 2023)	https://github.com/airtai/monotonic-nn?tab=readme-ov-file
Scalable monotonic neural network	SMNN	Partially connected exponentiated, ReLU, and confluence units, with ReLU-n activations on the monotonic pathways.	Partial	Exact	(Kim & Lee, 2024)	https://github.com/retna319/SMNN
Deep isotonic embedding network	DIEN	Order-preserving embeddings combined with monotone and non-monotone subnetworks.	Partial	Exact	(Zhao et al., 2024a)	–
Certified monotonic Kolmogorov–Arnold network	MonoKAN	Monotone cubic Hermite edge splines and sign-constrained weights enforced by post-update clamping.	Partial	Exact	(Polo-Molina et al., 2026b)	https://github.com/alejandropolo/MonoKAN

does not make the underlying network globally monotone by construction.

A prominent training-based family augments the predictive loss with derivative penalties evaluated at finitely many constraint points. However, PWL, MixupPWL, and UniformPWL do not use an identical penalty. Let M denote the coordinates required to be non-decreasing and define $D_M h(\mathbf{z}) = \sum_{j\in M} \partial h(\mathbf{z})/\partial z_j$. The original PWL penalty is

$$\Omega_{\mathrm{PWL}}(h, M) = \mathbb{E}_{\mathbf{z}\sim\hat{P}_{\mathrm{train}}}\left[\max\left\{0, -D_M h(\mathbf{z})\right\}\right], \tag{29}$$

an unsquared penalty on the aggregate negative divergence evaluated at empirical training points (Gupta et al., 2019). It therefore differs from a coordinatewise sign penalty. MixupPWL and UniformPWL instead use

$$\Omega_{\mathrm{Mixup}}(h, M) = \mathbb{E}_{\mathbf{z}\sim\nu_{\mathrm{mix}}}\left[\sum_{j\in M}\left(\max\left\{0, -\frac{\partial h(\mathbf{z})}{\partial z_j}\right\}\right)^2\right], \tag{30}$$

$$\Omega_{\mathrm{Uniform}}(h, M) = \mathbb{E}_{\mathbf{z}\sim\mathcal{U}(\mathcal{X})}\left[\sum_{j\in M}\left(\max\left\{0, b - \frac{\partial h(\mathbf{z})}{\partial z_j}\right\}\right)^2\right], \qquad b > 0. \tag{31}$$

For MixupPWL, ν_{mix} is obtained by augmenting a mini-batch with uniformly sampled points, randomly pairing elements of the combined

set, and evaluating convex interpolations; the pairs can therefore be data–data, data–uniform, or uniform–uniform (Monteiro et al., 2021, 2022). UniformPWL draws constraint points throughout a bounded input domain and imposes the positive derivative margin b (Liu et al., 2020). Thus, the three methods differ in both penalty construction and constraint-point sampling. Because each penalty is evaluated at finitely many points, a zero observed penalty is empirical evidence rather than a global certificate.

The Partial Monotonic Model (PMM) introduced by Trindade et al. (2021) is a hybrid architecture-and-loss formulation. Monotonic and unconstrained feature subsets are processed by separate subnetworks, whose representations are concatenated and passed through additional dense layers. Its soft-constraint variant combines the task loss with a gradient-based monotonicity penalty: p controls the relative contribution of the objectives, while a dynamically determined scale factor s balances their magnitudes. Because its dominant enforcement mechanism is loss-based, PMM is retained, but it is neither fully architecture-agnostic nor globally certified.

Another recent regularizer is Domain-Informed Monotonicity (DIM) (Salim et al., 2025). For each designated feature and current mini-batch, DIM fits a one-dimensional linear reference to the network's current predictions, sorts the predictions in ascending order, reorders that feature using the same permutation, and penalizes adjacent prediction increments that fall below the corresponding increments of the fitted reference. The reference is therefore estimated from each current batch rather than supplied as a fixed global baseline. DIM requires no architectural modification, but its batch-wise penalty is a soft empirical constraint and does not certify monotonicity over the complete input domain.

Overall, training-based monotonicity induction mechanisms offer substantial modeling flexibility. Many can be integrated into standard neural architectures, allowing practitioners to balance predictive performance with monotonicity requirements. Their effectiveness depends strongly on the regularization strength, the constraint-point sampling strategy, and the formulation used to quantify monotonicity violations. Because the constraints are typically evaluated at finitely many sampled points, these methods generally provide empirical monotonicity rather than a certified global guarantee. Nevertheless, they constitute a practically important family for incorporating domain knowledge through standard optimization procedures while retaining much of the flexibility of the underlying predictive model.

3.2.2. *Verification- and certification-based monotonicity mechanisms*

An important family focuses on verifying or certifying monotonicity for a specified model, prediction rule, and input domain, rather than merely encouraging monotonic behavior through sampled training penalties. These methods use optimization-based, logical, or analytic verification tools to establish sufficient conditions, rule out monotonicity counterexamples, or construct a certified monotone prediction rule. Depending on the mechanism, the certified object may be the network itself, its relevant partial derivatives, or a prediction-time envelope. These methods are particularly relevant to safety-critical and high-stakes applications where monotonic behavior requires formal guarantees rather than empirical encouragement, and they are closely connected to the broader field of neural network verification.

Bunel et al. (2018) cast verification of piecewise-linear neural networks as counterexample search and global optimization, showing that several existing solvers fit a common branch-and-bound framework. MILP encodings, SMT-based solvers such as Reluplex, and branch-and-bound methods based on convex relaxations are therefore different implementation routes (Katz et al., 2017; Rössig & Petkovic, 2021). Their relevance to monotonicity is that a violation can be formulated as a counterexample over a specified input domain.

Certified Monotonic Neural Networks (CeMNN) (Liu et al., 2020) certify partial monotonicity of piecewise-linear networks over a bounded domain. For a set M of coordinates required to be non-decreasing, define

$$U_M = \min_{\mathbf{x}\in\mathcal{X},\, j\in M} \frac{\partial f(\mathbf{x})}{\partial x_j}. \tag{32}$$

For a two-layer ReLU network, binary activation indicators convert this derivative minimization into an MILP. If the solver returns a valid certified lower bound $\underline{U}_M \le U_M$ with $\underline{U}_M \ge 0$, monotonicity on $\mathcal{X}$ is certified. CeMNN alternates sampled gradient regularization with MILP verification and increases the regularization coefficient when certification fails. Although a deep network can in principle be encoded in one MILP, the number of integer variables quickly becomes intractable. The original method therefore decomposes a deep network into two-layer modules and trains and verifies them separately. Monotonicity of every module is sufficient for monotonicity of their composition, but this test is incomplete: the complete network may be monotone even when some individual module cannot be certified.

Counterexample-Guided Learning of Monotonic Neural Networks (COMET) (Sivaraman et al., 2020) combines counterexample-guided training with a monotonic prediction envelope. ReLU networks and candidate violations are encoded in SMT with linear real arithmetic, whereas maximal counterexamples and envelope queries use Optimization Modulo Theories (OMT). During training, COMET generates maximal upper and lower envelope counterexamples relative to each training point and augments subsequent batches with pseudo-labeled counterexamples. For regression, the relevant current predictions are averaged to construct the pseudo-label; for classification, the counterexamples inherit the corresponding training label. At prediction time, the upper envelope for increasing-feature set M is

$$f^u_M(\mathbf{x}) = \max_{\mathbf{z}\in\mathcal{X}} \left\{ f(\mathbf{z}) : z_j \le x_j \ (j \in M), \quad z_j = x_j \ (j \notin M) \right\}, \tag{33}$$

with a dual lower envelope obtained by minimizing over inputs that dominate $\mathbf{x}$ in the monotonic coordinates. The certified guarantee belongs to this prediction-time envelope rather than to the unconstrained base network, and every corrected prediction requires a solver query.

LipVor (Polo-Molina et al., 2026a) provides a sufficient certificate for smooth, already-trained networks. On a compact domain $\mathcal{X}$, monotonicity in coordinate j is reduced to positivity of $g_j(\mathbf{x}) = \partial f(\mathbf{x})/\partial x_j$. A valid upper bound on the Lipschitz constant of g_j extends a positive derivative evaluation at each sampled site to a certified neighborhood. LipVor constructs a Voronoi tessellation and certifies $g_j(\mathbf{x}) > 0$ throughout $\mathcal{X}$ when every cell is covered by the certified neighborhood of its generator. The certificate therefore assumes Lipschitz-continuous partial derivatives, a valid Lipschitz upper bound, and a specified compact domain. The user-chosen $\varepsilon > 0$ serves as a finite-termination and counterexample threshold: after finitely many evaluations, the algorithm either satisfies the sufficient coverage condition and certifies strict positivity, or establishes that the function is not ε-positive by finding a value below ε. The latter outcome does not rule out strict positivity when the minimum lies between zero and ε. LipVor is therefore a sufficient certificate of strict partial monotonicity, not a complete decision procedure for ordinary non-decreasing monotonicity; moreover, the number of sites required for coverage can grow rapidly with dimension.

CeMNN, COMET, and LipVor avoid imposing one common globally monotone parameterization, but certify different objects under different assumptions: CeMNN certifies a bounded-domain piecewise-linear network, COMET certifies a prediction-time envelope, and LipVor certifies strict positivity of the relevant partial derivatives for a smooth network on a compact domain; in LipVor, ε controls finite termination and the negative outcome rather than the certified positive margin. They respectively inherit costs from mixed-integer optimization, logical/optimization queries, and geometric domain coverage. From a complexity-theoretic perspective, monotonicity by construction should be distinguished from post-hoc certification of an arbitrary trained network. Related results show that, for feed-forward ReLU networks with rational parameters, reachability under rational linear input and output specifications is NP-complete, whereas ReLU-network equivalence is co-

Table 2
Representative summary of non-structural monotonicity approaches.

Method	Abbreviation	Architecture	Core mechanism	Monotonicity assessment	Guarantee	Reference
Training-based mechanisms						
Monotonicity hint	MH	Architecture-agnostic	Soft monotonicity error incorporated into a Bayesian-inspired learning objective.	Empirical pairwise comparisons	Soft	(Abu-Mostafa, 1990; Sill & Abu-Mostafa, 1996)
Domain-Adapted Neural Networks	DANN	Architecture-compatible loss	Domain-rule loss with Boolean masks and ReLU penalties for reversed predicted orderings.	Empirical rule violations	Soft	(Muralidhar et al., 2018)
Point-wise gradient-penalty family	PWL variants	Differentiable networks	Unsquared aggregate-divergence penalty for PWL; coordinatewise squared penalties at mixup or uniform points for MixupPWL and UniformPWL, with a positive margin in the latter.	Aggregate divergence or coordinate-wise gradient signs at sampled points	Soft	(Gupta et al., 2019; Liu et al., 2020; Monteiro et al., 2021, 2022)
Partial Monotonic Model	PMM	Hybrid dual-stream network	Separate monotonic and unconstrained streams with a dynamically scale-adjusted monotonicity loss.	Gradient signs on empirical samples	Soft	(Trindade et al., 2021)
Domain-informed monotonicity	DIM	Architecture-compatible loss	Fits a batch-wise linear reference to current predictions and penalizes insufficient prediction increments.	Batch-wise reference comparison	Soft	(Salim et al., 2025)
Verification- and certification-based mechanisms						
Certified monotonic neural network	CeMNN	Piecewise-linear network	MILP derivative certification on a bounded domain; deep networks use two-layer module decomposition.	Certified derivative lower bound	Certified	(Liu et al., 2020)
Counterexample-guided learning of monotonic neural networks	COMET	ReLU-based neural network	Counterexample-guided training with an SMT/OMT prediction-time monotonic envelope.	SMT/OMT counterexample search	Certified	(Sivaraman et al., 2020)
Lipschitz–Voronoi certification	LipVor	Smooth neural network	Lipschitz derivative bounds and Voronoi coverage on a compact domain; ϵ controls finite termination rather than the certified margin.	Derivative positivity and Voronoi coverage	Certified	(Polo-Molina et al., 2026a)

NP-complete (Wurm, 2023); these results characterize broader neural-network verification tasks rather than monotonicity alone.

For monotonicity specifically, Wurm (2025) formalizes a monotonicity rule as preservation of an output class i for every pair $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ satisfying $A\mathbf{x} \leq A\mathbf{y}$, and a total-monotonicity rule as $N_i(\mathbf{x}) \leq N_i(\mathbf{y})$ for every such pair. For feed-forward ReLU networks with rational parameters, deciding whether the former rule holds globally is co-NP-complete for both standard and classifying networks; an analogous co-NP-complete result is obtained for Boolean networks, while linear-time reductions connect monotonicity, total-monotonicity, and oblique-rule verification. Thus, the complementary decision problem asks whether a violation exists and admits a polynomially checkable certificate, whereas global certification must rule out all such violations. Consequently, finite gradient sampling can reveal violations but cannot, by itself, certify their absence. These worst-case verification results help explain the scalability limitations of exact post-hoc MILP/SMT verification and provide an additional motivation for architectures that guarantee monotonicity by construction. In practice, the cost of exact certification also depends on the solver, network size, activation-pattern encoding, and specified input domain.

A related but distinct classical result concerns representational complexity. Monotone Boolean circuits provide monotonicity by construction using only AND and OR gates, yet classical lower bounds show that representing certain monotone functions in this way can require superpolynomial or exponential circuit size (Alon & Boppana, 1987; Razborov, 1985). These circuit-size results are not neural-network certification lower bounds; rather, they expose a parallel structural trade-off: syntactic monotonicity guarantees can make verification immediate while restricting expressive efficiency.

These verification-based approaches demonstrate how formal optimization, logical reasoning, and analytic or geometric certification frameworks can be used to establish monotonicity guarantees for specified networks, derivatives, or prediction rules, providing stronger statements than purely sampled regularization. However, this increased rigor can incur substantial computational overhead, limiting scalability to larger or deeper models. Reconciling expressive, large-scale training with efficient, certifiable monotonicity verification remains a core open challenge in this field.

3.2.3. Summary of non-structural monotonicity approaches

Non-structural approaches enforce or assess monotonicity primarily through objectives, verification procedures, or prediction-time correction. They often preserve more architectural flexibility than methods that guarantee monotonicity by construction, but they are not uniformly architecture-agnostic and can incur substantial certification or inference cost.

The reviewed methods range from soft empirical induction to formal certification. In Table 2, “Soft” denotes monotonicity encouraged through sampled training penalties without a domain-wide proof, whereas “Certified” denotes a formally established guarantee for the stated object and input domain under the specified assumptions.

3.3. Taxonomy summary and paradigm comparison

Sections 3.1 and 3.2 introduced two major paradigms for incorporating monotonicity constraints into neural networks. These two paradigms represent fundamentally different design philosophies. Structure-based methods embed monotonicity into the model architecture itself, typically providing strong guarantees but limiting architectural flexibility. Non-structural approaches instead act mainly through optimization, regularization, formal verification, or prediction-time correction. They often retain greater architectural flexibility, but this is not universal, and their guarantees and computational costs depend on the particular mechanism.

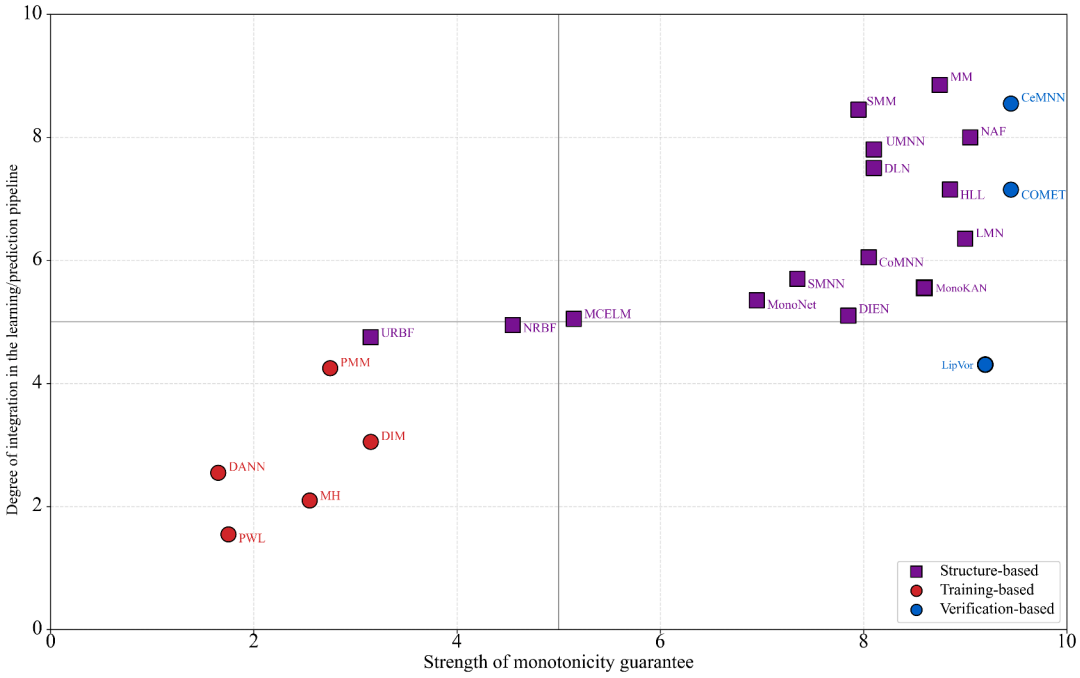


Fig. 7. Conceptual map of representative methods for monotonicity constraints in neural networks summarized in Tables 1 and 2. Structure-based approaches are represented by purple squares, non-structural approaches by circles, training-based methods by red circles, and verification and certification-based approaches by blue circles.

Fig. 7 illustrates the conceptual relationships among the representative methods discussed in Tables 1 and 2, providing a visual overview of how these approaches occupy different regions of the monotonicity design space. The figure is intended as a semi-qualitative conceptual map rather than a quantitative ranking plot. Specifically, the horizontal axis summarizes relative guarantee strength, together with the domain, assumptions, and object to which the guarantee applies, ranging from soft or empirical enforcement to exact or certified guarantees. The vertical axis reflects integration across the learning and prediction pipeline, not network architecture alone. The positions are therefore conceptual rather than precise numerical measurements.

4. Experimental comparison analysis

This section describes the experimental framework for empirically evaluating the effectiveness of applying representative monotonicity constraints in neural networks. We detail the experimental setup and process to ensure a fair comparison.

4.1. Experimental setup

The experimental setup is designed to assess both predictive performance and monotonicity behavior across a diverse collection of datasets and learning tasks. All experiments follow a common outer-evaluation and hyperparameter-selection protocol, with method-specific architectures and execution requirements.

4.1.1. Datasets

The experiments use 12 real-world tabular datasets. They span multiple application domains and include both regression and binary classification tasks, with substantial variation in sample size, dimensionality, and the number of monotonic features. Dataset sizes range from 303 to 60,021 observations, and the preprocessed inputs contain up to 280 features. Table 3 summarizes their sample sizes, fixed outer partitions, numbers of monotonic features, and task types, with "Raw/model inputs" denoting the numbers of selected predictive attributes before encoding and resulting model inputs after encoding, respectively. Below, we briefly describe each dataset.

- **Abalone:** A regression dataset from biology that aims to predict the age of abalone from physical measurements. Whole weight, shucked weight, viscera weight, and shell weight are assumed to be monotonically increasing with age (Nash et al., 1995).
- **Auto MPG:** A regression dataset for predicting automobile fuel efficiency. Vehicle weight, displacement, and horsepower are enforced as monotonically non-increasing with respect to miles per gallon (Quinlan, 1993).
- **Boston Housing:** A classic regression dataset for predicting median house prices in the Boston area. Among its features, the number of rooms exhibits a known increasing monotonic relationship with house value (Harrison & Rubinfeld, 1978).
- **COMPAS:** A binary classification dataset derived from criminal justice records in Florida. The task is to predict whether an individual will reoffend within two years. Four features, number of prior adult convictions, juvenile felony count, juvenile misdemeanor count, and number of other convictions, are assumed to be monotonically non-decreasing (Angwin et al., 2016).
- **ERA:** A regression dataset originating from an academic decision-making experiment that evaluates job candidates based on multiple attributes, all of which follow monotonic relationships with the final score (Ben-David et al., 1989).
- **ESL:** A regression dataset containing industrial job applicant profiles. Candidates are evaluated by expert psychologists using psychometric tests and interviews, producing an overall fitness score that is assumed to be monotonic with respect to all input attributes (Ben-David et al., 1989).
- **Heart Disease:** A binary classification dataset used to predict the presence or absence of heart disease. Resting blood pressure and cholesterol levels are assumed to have monotonic effects on disease likelihood (Janosi et al., 1988).
- **LEV:** A regression dataset consisting of anonymous lecturer evaluations from MBA courses. Students rated lecturers on several attributes prior to receiving final grades. All attributes are assumed to

Table 3
Characteristics of the 12 real-world datasets.

Dataset	Total	Train/Test	Raw/model inputs	Monotonic inputs	Task
Abalone	4177	3341/836	8/8	4	Regression
Auto MPG	392	313/79	7/7	3	Regression
Boston Housing	505	404/101	13/13	1	Regression
COMPAS	6172	4937/1235	7/7	4	Classification
ERA	1000	800/200	4/4	4	Regression
ESL	488	390/98	4/4	4	Regression
Heart Disease	303	242/61	13/13	2	Classification
LEV	1000	800/200	4/4	4	Regression
SWD	1000	800/200	10/10	7	Regression
Adult	48,842	32561/16281	14/108	2	Classification
Default Credit	30,000	24000/6000	23/33	6	Classification
BlogFeedback	60,021	52397/7624	280/280	4	Regression

contribute monotonically to the overall evaluation score (Ben-David et al., 1989).

- **SWD:** A regression dataset containing real-world risk assessments conducted by qualified social workers to evaluate whether children should remain with their families. These assessments are frequently used in judicial decision-making and follow established monotonic criteria (Ben-David et al., 1989).
- **Adult:** A binary classification dataset for predicting whether annual income exceeds USD 50000. The official training and test files are retained. Holding all encoded covariates fixed, education level (education-num) and capital gain are treated as non-decreasing domain assumptions for the probability of high income (Becker & Kohavi, 1996).
- **Default Credit:** A binary classification dataset for predicting default on credit-card payments. The six repayment-status variables (X6–X11) are treated as non-decreasing: a larger delay status is assumed not to reduce default risk (Yeh & Lien, 2009).
- **BlogFeedback:** A regression dataset for predicting the number of comments received in the following 24 hours. We retain the official temporal separation between the training file and all 60 test files. UCI features 51–54, which summarize total and recent comment counts before the prediction horizon, are treated as non-decreasing (Buza, 2014).

4.1.2. Evaluation criteria

Predictive performance is evaluated using the normalized Root Mean Squared Error (NRMSE) for regression tasks and error rate for binary classification tasks.

For regression tasks, predictive performance is measured using NRMSE, defined as

$$\mathrm{NRMSE} = \frac{\mathrm{RMSE}}{\max(y_{\mathrm{tr}}) - \min(y_{\mathrm{tr}})}, \tag{34}$$

where y_{tr} denotes the raw targets in the fixed outer-training partition. Predictions and targets are inverse-transformed to the original target scale before evaluation. This range-based normalization prevents test-target leakage and renders regression errors dimensionless.

For binary classification tasks, predictive performance is measured by the error rate, defined as

$$\text{Error Rate} = 1 - \text{Accuracy}. \tag{35}$$

A sample is assigned to the positive class when its model logit is greater than zero, equivalently when the corresponding sigmoid probability exceeds 0.5.

Empirical monotonicity violation rates are reported for the unconstrained MLP baseline and the three training-based monotonicity-induction methods, namely PWL, MixupPWL, and UniformPWL. A violation is recorded when at least one constrained feature satisfies $\partial f/\partial x_j < -10^{-8}$ at an evaluated point after all decreasing relationships have been converted to increasing ones. The violation rate is the proportion of evaluated points containing at least one such coordinatewise violation. Lower values indicate better adherence to the specified monotonic relationships, with zero corresponding to no detected violations under the given finite-sample audit. Structure-based approaches are not included in this empirical violation-rate comparison because their monotonicity properties follow from their architectural constructions under the corresponding stated assumptions.

For MLP, PWL, MixupPWL, and UniformPWL, monotonicity is evaluated post hoc under three complementary settings: (i) 1000 points drawn uniformly from the outer-training-normalized input cube $[0,1]^d$; (ii) up to 1000 points sampled without replacement from the outer-training partition; and (iii) up to 1000 points sampled without replacement from the fixed outer-test partition. If either observed-data partition contains fewer than 1000 observations, we use all available observations. We apply the same audit to all four models, and we do not use their results for parameter optimization or model selection. These settings distinguish behavior on observed training and test inputs from behavior across the normalized input domain.

4.1.3. Data preprocessing

The 12 datasets are processed using dataset-specific cleaning and encoding operations followed by a common transformation pipeline. Complete cases are used for Abalone, Auto MPG, Boston Housing, COMPAS, ERA, ESL, Heart Disease, LEV, and SWD, and no numerical imputation is applied. In Abalone, the "sex" attribute is integer-encoded; in Auto MPG, "car name" is removed; and COMPAS is filtered according to the criteria of Angwin et al. (2016), after which race and sex are integer-encoded. The Heart Disease target is retained in its supplied binary form. For Adult, missing categorical entries are assigned an explicit "Unknown" level, the eight categorical attributes are one-hot encoded using the vocabulary of the supplied training partition, and the income labels are mapped to the binary ≤ 50K and > 50K classes. For Default Credit, the identifier is omitted, while sex, education, and marital status are one-hot encoded using categories observed in the outer training partition. For BlogFeedback, all 60 supplied test files are concatenated to form the temporal test partition. These transformations produce 108 model inputs from the 14 raw Adult attributes and 33 model inputs from the 23 raw Default Credit attributes; BlogFeedback retains all 280 numerical inputs.

All specified monotonic relationships are represented in a non-decreasing direction: features with decreasing relationships are multiplied by -1, and the constrained features are placed before the remaining inputs. Integer- and one-hot-encoded categorical variables are treated as unconstrained. During hyperparameter search, feature scaling and regression-target standardization are fitted on the fit portion of the development subset and applied to its validation portion. During each final repetition, these transformations are fitted on the complete outer-training partition and applied to the outer-training and fixed outer-test partitions; the transformed outer-training data are then divided into model-fitting and early-stopping subsets. Predictions and targets are returned to the original scale for evaluation, and NRMSE is normalized by the range of the raw outer-training targets.

4.1.4. Training protocol

The empirical study comprises three complementary comparisons aligned with the taxonomy. The primary structure-based benchmark uses an unconstrained MLP (Hornik et al., 1989) as the predictive reference and compares eight monotonic predictors spanning the five architectural families described in Section 3.1: $\mathrm{MLP}_{\mathrm{wc}}$ represents a sign-constrained extension of a conventional feed-forward network; MM and SMM represent min–max architectures; HLL represents lattice-based architectures; UMNN represents flow-based architectures; and LMN, CoMNN, and SMNN represent expressiveness- and scalability-oriented architectures. As a targeted ablation within the min–max family, we additionally compare MM and SMM with variants that preprocess the unconstrained inputs using an auxiliary MLP, denoted $\mathrm{MM}_{\mathrm{aux}}$ and $\mathrm{SMM}_{\mathrm{aux}}$.

Table 4
Primary predictive metrics for structure-based monotonicity approaches.

Dataset	MLP	MLP_{wc}	MM	SMM	HLL	UMNN	LMN	CoMNN	SMNN
Abalone	**0.0760 ± 0.0005**	0.1120 ± 0.0105	0.0893 ± 0.0014	0.0850 ± 0.0002	0.0853 ± 0.0002	0.0851 ± 0.0004	0.0899 ± 0.0008	0.0850 ± 0.0005	0.0853 ± 0.0002
Auto MPG	0.0677 ± 0.0024	0.2265 ± 0.0144	0.0756 ± 0.0044	0.0717 ± 0.0094	0.0692 ± 0.0043	**0.0663 ± 0.0053**	0.0815 ± 0.0008	0.0704 ± 0.0056	0.0767 ± 0.0060
Boston Housing	0.0699 ± 0.0043	0.1181 ± 0.0304	0.0733 ± 0.0039	0.0662 ± 0.0032	**0.0656 ± 0.0028**	0.0796 ± 0.0071	0.0940 ± 0.0036	0.0749 ± 0.0087	0.0718 ± 0.0021
COMPAS	**0.2980 ± 0.0025**	0.3391 ± 0.0132	0.3038 ± 0.0038	0.3093 ± 0.0074	0.3171 ± 0.0030	0.2986 ± 0.0019	0.3292 ± 0.0031	0.3051 ± 0.0044	0.3017 ± 0.0039
ERA	0.1978 ± 0.0019	0.2337 ± 0.0015	0.2022 ± 0.0044	0.1976 ± 0.0016	**0.1974 ± 0.0025**	0.2003 ± 0.0026	0.1992 ± 0.0015	0.1989 ± 0.0008	0.2349 ± 0.0247
ESL	**0.0650 ± 0.0009**	0.2493 ± 0.0199	0.0661 ± 0.0005	0.0663 ± 0.0022	0.0675 ± 0.0017	0.1520 ± 0.0527	0.0675 ± 0.0008	0.0664 ± 0.0005	0.1637 ± 0.0137
Heart Disease	**0.1902 ± 0.0222**	0.3082 ± 0.0191	0.3836 ± 0.1810	0.2131 ± 0.0293	**0.1902 ± 0.0131**	0.2262 ± 0.0334	0.2820 ± 0.0123	0.2361 ± 0.0167	0.4557 ± 0.2168
LEV	0.1708 ± 0.0006	0.2318 ± 0.0148	0.1712 ± 0.0020	0.1717 ± 0.0011	0.1713 ± 0.0019	0.1707 ± 0.0006	**0.1704 ± 0.0009**	0.1718 ± 0.0009	0.2031 ± 0.0290
SWD	**0.2134 ± 0.0038**	0.2748 ± 0.0047	0.2200 ± 0.0015	0.2158 ± 0.0022	0.2146 ± 0.0022	0.2195 ± 0.0033	0.2162 ± 0.0018	0.2180 ± 0.0034	0.2251 ± 0.0080
Adult	0.1499 ± 0.0011	0.1474 ± 0.0006	0.1481 ± 0.0011	0.1482 ± 0.0012	0.1550 ± 0.0009	0.1476 ± 0.0013	0.1590 ± 0.0005	**0.1446 ± 0.0017**	0.1509 ± 0.0064
Default Credit	0.1824 ± 0.0010	0.1975 ± 0.0016	0.1831 ± 0.0018	0.1819 ± 0.0022	0.1931 ± 0.0009	0.1814 ± 0.0014	0.2002 ± 0.0010	**0.1801 ± 0.0011**	0.1887 ± 0.0073
BlogFeedback	0.0197 ± 0.0007	0.0214 ± 0.0000	0.0182 ± 0.0006	0.0189 ± 0.0007	0.0207 ± 0.0007	0.0185 ± 0.0007	0.0183 ± 0.0004	**0.0178 ± 0.0008**	0.0188 ± 0.0005

The training-based study evaluates PWL, MixupPWL, and UniformPWL at five fixed regularization strengths, $\lambda = 10^q$ for $q \in \{0, 1, 2, 3, 4\}$, as representatives of the training-based non-structural approaches discussed in Section 3.2.1. These methods share the same ReLU MLP backbone family and hyperparameter search space while differing in their derivative penalties and constraint-point constructions, enabling a coherent comparison under a common supervised tabular-learning protocol.

Other training-based approaches are discussed qualitatively because they depart from this shared setting. MH requires separately supplied monotonicity-hint pairs, DANN relies on explicit domain rules, PMM combines a specialized dual-stream architecture with a dynamically adjusted loss, and DIM constructs a batch-wise reference from the model's current predictions. Their inclusion would therefore require different supervision, architectural assumptions, or constraint definitions.

The verification- and certification-based approaches in Section 3.2.2 also require separate evaluation procedures. The benchmark includes CeMNN's sampled-gradient component through UniformPWL but not its MILP certification stage; COMET uses SMT/OMT searches to construct a prediction-time monotonic envelope, whereas LipVor combines derivative Lipschitz bounds with Voronoi coverage. These procedures certify different objects and require method-specific computational budgets, so they are evaluated qualitatively rather than within the common end-to-end benchmark.

Adult and BlogFeedback retain their supplied train–test partitions, while the remaining ten datasets use a fixed 80/20 outer split with seed 42, stratified for binary classification. The outer-test partition is reserved for final evaluation. Hyperparameter selection, fitted numerical scaling and regression-target standardization, train-vocabulary one-hot encoding where applicable, model fitting, and early stopping use only the outer-training data.

An independent hyperparameter search is conducted for every evaluated configuration: each method–dataset pair in the structure-based and auxiliary-network comparisons, and each regularizer–dataset–λ triplet in the training-based study. A seeded 20% development subset is drawn from the outer training partition, stratified for classification. Hyperparameters are optimized on this subset using a fixed 80/20 fit–validation split, Optuna's Tree-structured Parzen Estimator sampler, and 20 trials, each trained for at most 20 epochs. Trials and checkpoints are selected by RMSE on standardized targets for regression and by error rate for classification. After configuration selection, five final training repetitions are performed with seeds 42–46. Each repetition independently divides the outer training partition into 80% model-fitting and 20% early-stopping subsets, again stratified for classification, and trains for at most 100 epochs with patience 10. The selected checkpoint is then evaluated on the fixed outer test partition. Reported means and standard deviations therefore reflect variability across the five training seeds and their associated early-stopping splits, rather than variability across cross-validation folds.

All models are implemented in PyTorch and trained using the schedule-free AdamW optimizer (Defazio et al., 2024); the optimizer is switched between its training and evaluation parameter sequences before optimization and validation/checkpointing, respectively. GPU-capable implementations are run with CUDA 12.8. HLL runs on the CPU because of the execution requirements of the employed pmlayer implementation, whereas UMNN runs on CUDA. To support standardized regression targets and binary-classification logits, the bounded HLL lattice output is followed by a numerically clamped logit link; this transformation is non-decreasing and therefore preserves monotonicity. For LMN, λ is fixed at 1 and is used both as the Lipschitz bound of the base network and as the coefficient of the monotonic residual.

PWL, MixupPWL, and UniformPWL use the same ReLU MLP backbone family and hyperparameter search space as MLP. PWL applies the original negative-divergence penalty on empirical mini-batches. MixupPWL augments each mini-batch with an equal number of points sampled uniformly from the scaled input domain, forms up to 1024 inter-

polants from unordered pairs of the combined set by randomly subsampling pairs when necessary, and evaluates its coordinatewise squared-gradient penalty at these points. UniformPWL draws a fresh sample of 1024 points from the scaled input domain at every optimization step and uses the positive derivative margin $b = 0.2$. For MLP_{wc}, only first-layer columns associated with the designated monotonic features are constrained to be non-negative, while all downstream weights are non-negative; first-layer weights associated with unconstrained features remain unrestricted.

The architectures retain their method-specific capacity profiles because the monotonicity mechanisms introduce different structural components. Every evaluated configuration receives the same 20-trial search budget and final-training schedule, while trainable parameter counts for the selected structure-based and auxiliary-network configurations are reported separately. Predictive performance can therefore be interpreted jointly with model complexity without requiring identical parameterizations across architectural families.

4.1.5. Statistical analysis

To compare model performance across multiple datasets, we follow the non-parametric framework of Demšar (2006). Each method contributes one mean primary performance value per dataset (NRMSE for regression and error rate for classification; both lower is better). The Friedman test (Friedman, 1937) assesses systematic differences among methods using within-dataset ranks rather than raw metric values.

When the Friedman test rejects the null hypothesis, we apply the Nemenyi post-hoc test (Nemenyi, 1963) to all method pairs while controlling for multiple comparisons. For the structure-based comparison, average ranks and Nemenyi relationships are also summarized in a critical-difference (CD) diagram.

We additionally report two-sided Wilcoxon signed-rank tests (Wilcoxon, 1992) on the paired dataset-level primary metric values as exploratory pairwise comparisons. Because these pooled comparisons combine NRMSE for regression with error rate for classification, and because their p-values are not adjusted for multiple comparisons, they are interpreted only as supplementary evidence. The rank-based Friedman test and, when applicable, the multiplicity-controlled Nemenyi comparisons provide the primary basis for cross-dataset conclusions.

4.2. Experimental results

This section compares representative monotonicity mechanisms in terms of predictive performance, monotonicity satisfaction, and model complexity across methods and datasets.

4.2.1. Predictive performance and statistical comparison of structure-based monotonicity approaches

Table 4 gives the dataset-level predictive metrics. MLP has the lowest reported mean on Abalone, COMPAS, ESL, and SWD and shares the lowest reported mean with HLL on Heart Disease at the displayed precision. Among monotonic architectures, CoMNN has the lowest reported mean on Adult, Default Credit, and BlogFeedback; HLL does so on Boston Housing, ERA, and Heart Disease; UMNN does so on Auto MPG; and LMN does so on LEV. Hence, no monotonic architecture dominates uniformly. MLP_{wc} has the second-lowest reported mean on Adult but records the worst reported mean on eight of the 12 datasets.

Across the 12 datasets, the Friedman test rejects the null hypothesis of no systematic difference in within-dataset ranks among the nine methods ($\chi_F^2(8) = 35.9833$, $p = 1.77 \times 10^{-5}$). Fig. 8 summarizes the corresponding average ranks, with lower values indicating better predictive performance. MLP achieves the best (lowest) average rank (2.79), followed by CoMNN (3.79), SMM (3.88), UMNN (4.00), HLL (4.54), MM (5.08), LMN (6.21), SMNN (6.63), and MLP_{wc} (8.08). Thus, MLP ranks first overall, while CoMNN, SMM, and UMNN are the three top-ranked monotonic architectures. These ranks concern predictive performance only and are not normalized for model size.

At $\alpha = 0.05$, Table 5 reports the Nemenyi post-hoc p-values adjusted for all pairwise method comparisons. MLP has a significantly better average rank than MLP_{wc} ($p = .0001$) and SMNN ($p = .0176$). MLP_{wc} has a significantly worse average rank than CoMNN ($p = .0039$), SMM ($p = .0053$), UMNN ($p = .0080$), and HLL ($p = .0410$). No significant difference is detected for any other pair; the smallest remaining p-value is for MLP versus LMN ($p = .0572$). The overlapping non-significant groups in Fig. 8 therefore indicate that most methods cannot be statistically distinguished by the Nemenyi procedure on this benchmark.

The supplementary exploratory Wilcoxon analysis in Table 6 reports unadjusted two-sided pairwise comparisons. For MLP, nominally significant results are obtained against MLP_{wc} ($p = .0015$), MM ($p = .0161$), HLL ($p = .0244$), LMN ($p = .0034$), and SMNN ($p = .0010$), with the observed paired differences favoring MLP. No nominally significant difference is detected between MLP and SMM ($p = .1514$), UMNN ($p = .1294$), or CoMNN ($p = .1099$). For MLP_{wc}, the observed paired differences favor MLP, MM, SMM, HLL, UMNN, LMN, and CoMNN, and all corresponding tests are nominally significant. Among the remaining nominally significant comparisons between monotonic architectures, the observed paired differences favor CoMNN over MM, LMN, and SMNN; SMM over LMN and SMNN; MM and UMNN over SMNN; and HLL over LMN. Because these p-values are not adjusted for multiple comparisons, the Wilcoxon results are treated as complementary exploratory evidence rather than as the primary basis for cross-dataset conclusions.

Model complexity provides an additional perspective on the predictive results. As shown in Table 7, the selected parameter counts vary substantially across both methods and datasets, and no method is uniformly the most compact. UMNN and SMNN generally use larger configurations, whereas MLP_{wc}, LMN, and CoMNN are often among the more compact alternatives. HLL exhibits particularly strong dataset-dependent variation, ranging from very small configurations on some datasets to substantially larger ones on others. Taken together with Table 4, these results reveal no consistent relationship between model size and predictive performance: both compact and larger architectures achieve competitive results. MLP_{wc} further illustrates this point, because its generally weak predictive performance cannot be attributed uniformly to reduced model size. This pattern is consistent with the possibility that restrictive sign constraints limit useful representations. Overall, predictive performance and model complexity should be considered jointly when comparing structure-based monotonicity architectures.

Overall, the Friedman test detects a statistically significant overall difference among methods based on their within-dataset ranks, whereas the Nemenyi post-hoc analysis detects significant differences for only a small subset of method pairs. Among the monotonic architectures, CoMNN achieves the best observed average rank for predictive performance, followed by SMM, UMNN, and HLL. These four methods provide exact monotonicity guarantees by construction within their stated scopes, and the Nemenyi analysis detects no significant differences among them; moreover, none is uniformly best across datasets. MLP_{wc} has the worst observed average rank for predictive performance, suggesting that the performance of a simple sign-constrained baseline should not be taken as representative of monotonic architectures more broadly. Because the selected configurations vary substantially in parameter count across methods and datasets, interpret these results as comparisons of complete, independently tuned configurations rather than parameter-matched estimates of architectural superiority.

4.2.2. Effect of auxiliary networks for non-monotonic features in min–max architectures

We further investigate auxiliary networks that preprocess non-monotonic features, comparing MM with MM_{aux} and SMM with SMM_{aux}. Table 8 reports the corresponding dataset-level predictive metrics. MM_{aux} has a lower reported mean than MM on five datasets and a higher reported mean on seven, while SMM_{aux} has a lower reported mean than SMM on seven datasets and a higher reported mean on five. The magnitude of the changes also varies: MM_{aux} has a markedly lower reported

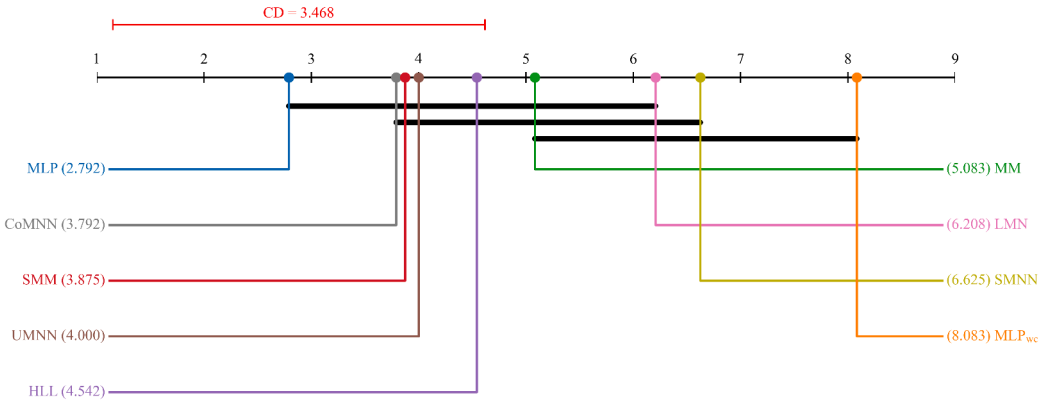


Fig. 8. CD diagram for the structure-based comparison.

Table 5
Nemenyi post-hoc p-values adjusted for all pairwise comparisons of structure-based monotonicity approaches.

	MLP	MLP$_{wc}$	MM	SMM	HLL	UMNN	LMN	CoMNN	SMNN
MLP	1.0000	0.0001	0.5083	0.9886	0.8238	0.9771	0.0572	0.9933	0.0176
MLP$_{wc}$	0.0001	1.0000	0.1536	0.0053	0.0410	0.0080	0.7608	0.0039	0.9304
MM	0.5083	0.1536	1.0000	0.9771	0.9999	0.9886	0.9854	0.9655	0.9060
SMM	0.9886	0.0053	0.9771	1.0000	0.9996	1.0000	0.4822	1.0000	0.2519
HLL	0.8238	0.0410	0.9999	0.9996	1.0000	0.9999	0.8603	0.9991	0.6392
UMNN	0.9771	0.0080	0.9886	1.0000	0.9999	1.0000	0.5608	1.0000	0.3133
LMN	0.0572	0.7608	0.9854	0.4822	0.8603	0.5608	1.0000	0.4311	1.0000
CoMNN	0.9933	0.0039	0.9655	1.0000	0.9991	1.0000	0.4311	1.0000	0.2155
SMNN	0.0176	0.9304	0.9060	0.2519	0.6392	0.3133	1.0000	0.2155	1.0000

Table 6
Exploratory two-sided Wilcoxon signed-rank p-values for structure-based monotonicity approaches.

	MLP	MLP$_{wc}$	MM	SMM	HLL	UMNN	LMN	CoMNN	SMNN
MLP	NaN	0.0015	0.0161	0.1514	0.0244	0.1294	0.0034	0.1099	0.0010
MLP$_{wc}$	0.0015	NaN	0.0269	0.0010	0.0024	0.0010	0.0049	0.0005	0.0640
MM	0.0161	0.0269	NaN	0.1514	0.7910	0.1758	0.3394	0.0425	0.0425
SMM	0.1514	0.0010	0.1514	NaN	0.6772	0.5693	0.0068	0.8477	0.0122
HLL	0.0244	0.0024	0.7910	0.6772	NaN	0.7910	0.0098	0.9697	0.0830
UMNN	0.1294	0.0010	0.1758	0.5693	0.7910	NaN	0.2036	0.6221	0.0093
LMN	0.0034	0.0049	0.3394	0.0068	0.0098	0.2036	NaN	0.0161	0.5186
CoMNN	0.1099	0.0005	0.0425	0.8477	0.9697	0.6221	0.0161	NaN	0.0088
SMNN	0.0010	0.0640	0.0425	0.0122	0.0830	0.0093	0.5186	0.0088	NaN

Table 7
Number of trainable parameters in the selected configurations for structure-based monotonicity approaches.

Dataset	MLP	MLP$_{wc}$	MM	SMM	HLL	UMNN	LMN	CoMNN	SMNN
Abalone	2465	153	288	1153	5073	7299	153	**77**	17,729
Auto MPG	705	145	**64**	385	824	19,087	225	229	17,665
Boston Housing	5121	**193**	2240	561	2978	5591	**193**	1241	35,137
COMPAS	**417**	705	768	769	9681	1939	1041	581	34,177
ERA	121	121	320	201	**16**	6939	121	77	4513
ESL	2433	121	100	201	**16**	339	201	145	4513
Heart Disease	1425	**193**	336	449	3145	8003	801	1045	801
LEV	1377	121	100	201	**16**	22,899	201	53	17,217
SWD	753	**169**	220	353	10,560	72,735	465	289	8993
Adult	7505	**953**	6976	4361	1009	33,899	**953**	7957	2041
Default Credit	833	1361	2720	1089	27,929	10,555	1633	**781**	2129
BlogFeedback	**2889**	4785	13,488	11,241	19,040	110,899	18,513	5825	52,545

mean than MM on Heart Disease, whereas it has higher reported means on SWD and Adult. These mixed directions suggest that the usefulness of an auxiliary network depends on both the backbone and the dataset rather than following a universal pattern.

The four-method Friedman test does not reject the null hypothesis of no systematic difference in within-dataset ranks among the four methods ($\chi_F^2(3) = 3.4615$, $p = .3258$). Accordingly, no Nemenyi post-hoc analysis is conducted. Table 9 reports exploratory, unadjusted, two-sided Wilcoxon signed-rank tests for the six method pairs. No nominally significant pairwise difference is detected between MM and MM$_{aux}$ ($p = 1.0000$) or between SMM and SMM$_{aux}$ ($p = .4575$), providing no evidence that auxiliary preprocessing systematically improves either

Table 8
Primary predictive metrics for min–max architectures with and without auxiliary networks.

Dataset	MM	MM_{aux}	SMM	SMM_{aux}
Abalone	0.0893 ± 0.0014	0.0852 ± 0.0003	**0.0850 ± 0.0002**	0.0855 ± 0.0002
Auto MPG	0.0756 ± 0.0044	0.0828 ± 0.0114	0.0717 ± 0.0094	**0.0677 ± 0.0037**
Boston Housing	0.0733 ± 0.0039	0.0773 ± 0.0104	**0.0662 ± 0.0032**	0.0667 ± 0.0040
COMPAS	0.3038 ± 0.0038	**0.3009 ± 0.0047**	0.3093 ± 0.0074	0.3067 ± 0.0095
ERA	0.2022 ± 0.0044	0.1989 ± 0.0006	**0.1976 ± 0.0016**	0.1991 ± 0.0017
ESL	0.0661 ± 0.0005	0.0662 ± 0.0007	0.0663 ± 0.0022	**0.0647 ± 0.0005**
Heart Disease	0.3836 ± 0.1810	0.2230 ± 0.0266	0.2131 ± 0.0293	**0.1934 ± 0.0191**
LEV	0.1712 ± 0.0020	0.1717 ± 0.0008	0.1717 ± 0.0011	**0.1709 ± 0.0013**
SWD	0.2200 ± 0.0015	0.2261 ± 0.0044	**0.2158 ± 0.0022**	0.2177 ± 0.0014
Adult	**0.1481 ± 0.0011**	0.1494 ± 0.0011	0.1482 ± 0.0012	0.1492 ± 0.0019
Default Credit	0.1831 ± 0.0018	**0.1817 ± 0.0015**	0.1819 ± 0.0022	**0.1817 ± 0.0007**
BlogFeedback	**0.0182 ± 0.0006**	0.0187 ± 0.0004	0.0189 ± 0.0007	0.0187 ± 0.0004

Table 9
Exploratory two-sided Wilcoxon signed-rank p-values for min–max architectures with and without auxiliary networks.

	MM	MM_{aux}	SMM	SMM_{aux}
MM	NaN	1.0000	0.1514	0.0327
MM_{aux}	1.0000	NaN	0.1201	0.0898
SMM	0.1514	0.1201	NaN	0.4575
SMM_{aux}	0.0327	0.0898	0.4575	NaN

Table 10
Number of trainable parameters in the selected min–max configurations with and without auxiliary networks.

Dataset	MM	MM_{aux}	SMM	SMM_{aux}
Abalone	**288**	2689	1153	674
Auto MPG	**64**	577	385	706
Boston Housing	2240	1793	**561**	1458
COMPAS	768	2369	769	**386**
ERA	320	449	**201**	330
ESL	**100**	189	201	330
Heart Disease	**336**	1169	449	2626
LEV	**100**	529	201	330
SWD	**220**	1025	353	674
Adult	6976	41,793	**4361**	20,866
Default Credit	2720	2673	**1089**	5122
BlogFeedback	13,488	35,777	**11241**	29,034

base architecture. Among all six pairwise comparisons, only MM versus SMM_{aux} has an unadjusted p-value below 0.05 ($p = .0327$), with the observed paired differences favoring SMM_{aux}. However, because the omnibus Friedman test is not significant and the Wilcoxon p-values are not adjusted for multiple comparisons, this isolated result is treated as exploratory rather than confirmatory. Overall, the statistical analysis does not establish a systematic predictive benefit from auxiliary preprocessing.

Table 10 shows that auxiliary preprocessing also changes model size in a non-uniform way. It increases the selected parameter count on 10 of the 12 datasets in both architectural families, sometimes by a large factor. The MM_{aux} is smaller only on Boston Housing and Default Credit, whereas SMM_{aux} is smaller only on Abalone and COMPAS. These capacity changes do not align consistently with the predictive changes in Table 8.

In summary, the non-significant Friedman result and the exploratory unadjusted two-sided Wilcoxon comparisons do not provide evidence of a statistically reliable aggregate benefit from auxiliary preprocessing. Because auxiliary preprocessing can also substantially increase model size in some configurations, evaluate it on a case-by-case basis rather than treating it as a generally beneficial extension.

4.2.3. *Effect of regularization strength on monotonicity induction*

We analyze PWL, MixupPWL, and UniformPWL, with an unregularized MLP included as the reference. Each regularized method is evaluated at five fixed regularization strengths, $\lambda \in \{10^0, 10^1, 10^2, 10^3, 10^4\}$. These settings span a broad range of nominal penalty weights, allowing the observed trade-offs between predictive performance and empirical monotonicity violations to be characterized without assuming a monotonic response to λ. Table 11 reports the primary predictive metric, while Tables 12–14 report empirical violation rates on random-domain points, points sampled from the outer-training partition, and points sampled from the fixed outer-test partition, respectively. The three rates are reported separately because agreement in one evaluation region does not imply agreement in another. Fig. 9 provides task-separated views of predictive performance versus the outer-test violation rate.

Table 11 shows that at least one regularized configuration has a lower reported primary-metric mean than MLP on 11 datasets; on ERA, the lowest regularized mean matches that of MLP at the displayed precision. Among the three regularized methods, PWL participates in the lowest reported primary-metric mean on seven datasets, whereas MixupPWL and UniformPWL each participate on four when ties at the displayed precision are retained. The lowest mean among the regularized configurations is attained at $\lambda \leq 10^2$ on 11 datasets; LEV is the only exception, with the minimum shared by MixupPWL at $\lambda = 10^3$ and UniformPWL at $\lambda = 10^4$. These results are descriptive comparisons of reported means rather than significance tests among the individual λ configurations.

Tables 12–14 report monotonicity-violation rates in three complementary evaluation regions. In the random-domain audit, UniformPWL participates in the lowest reported violation rate at one or more explored values of λ on every dataset when ties at the displayed precision are retained. This pattern is descriptively consistent with its use of uniformly sampled constraint points. In the outer-training audit, MixupPWL participates in the lowest reported rate at one or more explored values of λ on every dataset. The outer-test audit shows the same method-level pattern, with MixupPWL again achieving the lowest reported rate on every dataset; the outer-training and outer-test rates are also often descriptively similar. Nevertheless, random-domain and data-supported audits can differ substantially, as illustrated by some UniformPWL configurations on Abalone and Default Credit, where low random-domain violation rates do not correspond to similarly low outer-test rates. The three audits are therefore complementary empirical diagnostics rather than interchangeable estimates or formal global certificates.

Considering Table 11 together with Table 14, several regularized configurations are numerically better than MLP on both reported objectives, achieving both a lower primary predictive metric and a lower outer-test violation rate. On Auto MPG, UniformPWL with $\lambda = 10^0$ has a lower reported NRMSE than MLP (0.0632 versus 0.0677) and a lower outer-test violation rate (0.0177 versus 0.7949). On Heart Disease, PWL with $\lambda = 10^1$ has a lower reported error rate than MLP (0.1836 versus 0.1902) and a lower outer-test violation rate (0.2197 versus 0.8984).

Table 11
Primary predictive metrics across regularization strengths λ.

Dataset	MLP	PWL					MixupPWL					UniformPWL				
		10^0	10^1	10^2	10^3	10^4	10^0	10^1	10^2	10^3	10^4	10^0	10^1	10^2	10^3	10^4
Abalone	0.0760	**0.0759**	0.0769	0.0771	0.0779	0.0817	0.0843	0.0839	0.0844	0.0847	0.0849	0.0838	0.0847	0.0864	0.0870	0.0894
Auto MPG	0.0677	0.0713	0.0648	0.0703	0.0693	0.0750	0.0752	0.0643	0.0691	0.0714	0.0660	**0.0632**	0.0670	0.0689	0.0674	0.0675
Boston Housing	0.0699	0.0686	0.0708	0.0766	0.0749	0.0769	0.0753	0.0670	**0.0668**	0.0711	0.0733	0.0703	0.0722	0.0720	0.0687	0.0708
COMPAS	0.2980	0.3014	**0.2978**	0.3006	0.2998	0.3091	0.3004	0.2983	0.2996	0.3025	0.3553	0.3045	0.3012	0.3127	0.3304	0.3427
ERA	**0.1978**	0.1983	0.1988	0.1981	0.1985	0.1996	0.1982	0.1980	0.1986	0.1988	0.1984	0.1997	0.1993	**0.1978**	0.2014	0.1992
ESL	0.0650	0.0652	0.0660	0.0652	0.0654	0.0653	0.0663	**0.0649**	0.0661	0.0673	0.0651	0.0655	0.0652	0.0656	0.0657	0.0662
Heart Disease	0.1902	0.2033	**0.1836**	0.1902	0.1967	0.1902	0.2033	0.1967	0.2656	0.2361	0.2623	0.2066	0.2459	0.2885	0.2787	0.2689
LEV	0.1708	0.1719	0.1717	0.1715	0.1709	0.1701	0.1702	0.1709	0.1703	**0.1699**	0.1703	0.1716	0.1709	0.1705	0.1708	**0.1699**
SWD	0.2134	0.2137	0.2148	**0.2133**	0.2144	0.2144	0.2143	0.2157	0.2160	0.2158	0.2180	0.2181	0.2169	0.2213	0.2285	0.2280
Adult	0.1499	0.1494	**0.1474**	0.1489	0.1492	0.1553	0.1478	0.1483	0.1480	0.1490	0.1507	0.1477	0.1506	0.1479	0.1504	0.1503
Default Credit	0.1824	0.1828	**0.1814**	0.1830	0.1816	0.1859	0.1825	0.1822	0.1821	0.1825	0.1822	0.1829	0.1830	0.1840	0.1828	0.1829
BlogFeedback	0.0197	**0.0191**	0.0192	0.0193	0.0193	0.0193	0.0194	**0.0191**	0.0192	0.0192	0.0195	**0.0191**	0.0192	0.0196	0.0195	0.0202

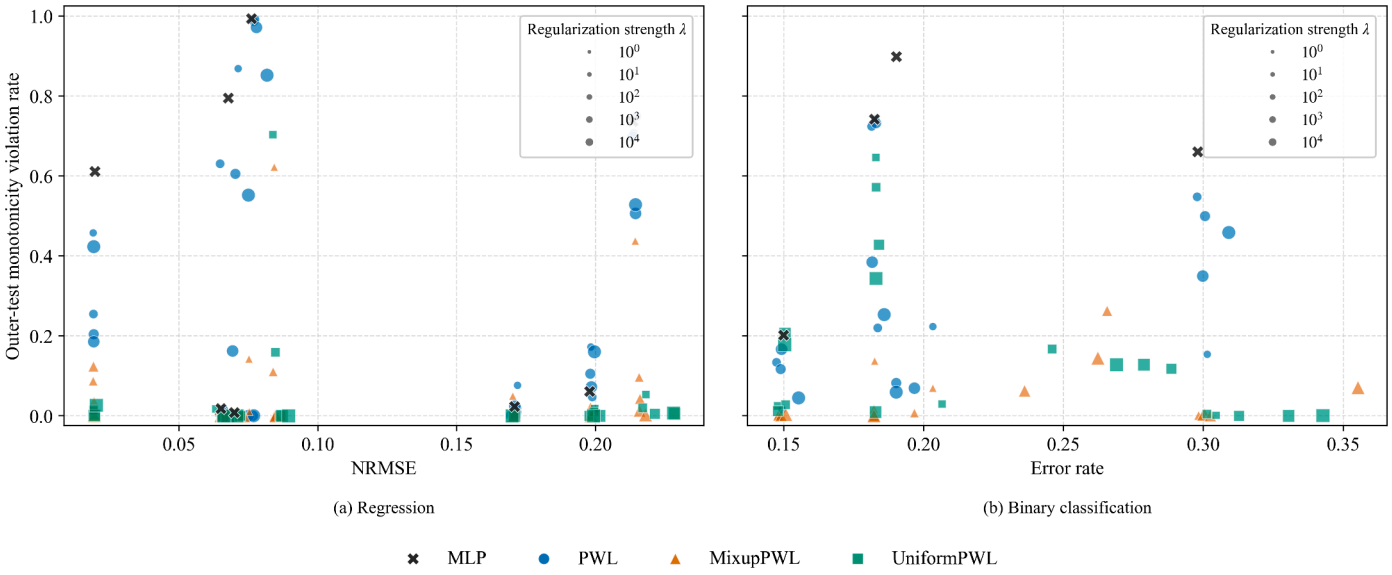


Fig. 9. Predictive performance–monotonicity trade-offs on the fixed outer test sets. MLP contributes one point per dataset, whereas each regularized method contributes five points per dataset for $\lambda \in \{10^0, 10^1, 10^2, 10^3, 10^4\}$; marker size increases with λ. Lower values are preferred on both axes. The pooled panels are descriptive summaries, and distances between points from different datasets are not interpreted as paired effect sizes.

BlogFeedback illustrates why the two objectives must be considered jointly: PWL at $\lambda = 10^0$, MixupPWL at $\lambda = 10^1$, and UniformPWL at $\lambda = 10^0$ share the same reported NRMSE at the displayed precision (0.0191), but their outer-test violation rates are 0.4574, 0.0864, and 0.0136, respectively.

Across Tables 12, 13, and 14, larger values of λ are often associated with lower reported violation rates, although the observed trajectories are not monotonic. MixupPWL and UniformPWL more often achieve the lowest reported violation rates when ties at the displayed precision are retained. This non-monotonic response may partly reflect that each value of λ corresponds to a separately tuned and trained model and therefore remains subject to optimization variability. On COMPAS, for example, Tables 11 and 14 show that MixupPWL at $\lambda = 10^2$ has a reported error rate of 0.2996 and no detected violations in the finite outer-test audit. In contrast, the $\lambda = 10^4$ configuration has numerically worse reported values for both the error rate and violation rate (0.3553 and 0.0704, respectively). Larger penalty coefficients therefore do not necessarily produce a better observed predictive–monotonicity trade-off.

A single-score comparison of the regularization strategies is complicated by the joint choice of method and λ. We therefore retain the primary predictive metric and violation rate as separate minimization objectives in a Pareto analysis. For each dataset, numerical value of λ, and evaluation region, one regularized method dominates another if it has values no greater on both objectives and a strictly lower value on at least one objective; a method is non-dominated if no competing regularized method dominates it. The unregularized MLP is not included in this matched comparison because it has no corresponding value of λ. The 12 datasets and five values of λ yield 60 matched dataset–λ conditions for each evaluation region. Because the three penalties differ in form and scale, equal numerical values of λ do not imply equal effective regularization; the resulting frequencies are therefore descriptive summaries conditional on the selected grid. Multiple methods may be non-dominated within one condition, and when methods have identical values on both objectives at the displayed precision, all tied methods are retained.

Table 15 shows that UniformPWL has the highest observed random-domain non-dominance rate (68.33%), narrowly exceeding MixupPWL (66.67%). MixupPWL has the highest observed outer-training and outer-test rates, at 73.33% and 75.00%, respectively. The corresponding rates for PWL are lower, at 43.33%, 46.67%, and 43.33%, even though PWL participates in the dataset-level best (lowest) reported primary-metric mean more frequently. Thus, PWL is more frequently associated with the best observed predictive result, whereas MixupPWL and UniformPWL more frequently appear on the within-condition prediction–monotonicity Pareto front. These frequencies are descriptive rather than inferential and do not establish statistically significant differences among the methods.

Table 12
Mean monotonicity violation rates on random-domain points over five final training repetitions (lower is better).

Dataset	MLP	PWL					MixupPWL					UniformPWL				
		10^0	10^1	10^2	10^3	10^4	10^0	10^1	10^2	10^3	10^4	10^0	10^1	10^2	10^3	10^4
Abalone	0.9512	0.9326	0.7952	0.9856	0.8280	0.9314	0.5086	0.0260	0.0028	0.0022	0.0004	0.0254	0.0004	**0.0000**	**0.0000**	**0.0000**
Auto MPG	0.7564	0.8230	0.6358	0.6338	0.1102	0.5492	0.1512	0.0176	0.0092	0.0016	**0.0000**	0.0020	**0.0000**	**0.0000**	**0.0000**	**0.0000**
Boston Housing	0.0104	0.0100	0.0024	**0.0000**	**0.0000**	0.0012	0.0234	0.0006	0.0004	0.0002	**0.0000**	0.0026	0.0002	**0.0000**	**0.0000**	**0.0000**
COMPAS	0.7900	0.5844	0.7070	0.4468	0.4398	0.5608	0.0056	0.0012	**0.0000**	**0.0000**	0.1344	**0.0000**	**0.0000**	0.0002	**0.0000**	**0.0000**
ERA	0.0684	0.1456	0.0684	0.1372	0.0676	0.1282	0.0376	0.0078	0.0106	0.0002	**0.0000**	0.0314	0.0046	**0.0000**	0.0002	**0.0000**
ESL	0.0406	0.0346	0.0668	0.0688	0.0512	0.0520	0.0082	0.0102	0.0064	0.0012	**0.0000**	0.0048	**0.0000**	0.0020	**0.0000**	**0.0000**
Heart Disease	0.6288	0.2046	0.3240	0.0684	0.1162	0.1168	0.0154	0.0064	0.2680	0.0224	0.1484	**0.0004**	0.1456	0.1084	0.1378	0.1404
LEV	0.0286	0.0528	0.0182	0.0070	0.0158	**0.0000**	0.0486	0.0024	0.0028	**0.0000**	0.0004	0.0010	0.0004	0.0002	**0.0000**	**0.0000**
SWD	0.6830	0.6438	0.7078	0.6874	0.4790	0.4626	0.3390	0.0504	0.0090	0.0030	**0.0002**	0.0142	0.0010	**0.0002**	0.0006	0.0008
Adult	0.1958	**0.0000**	0.0248	0.0026	0.0016	0.1574	**0.0000**	**0.0000**	**0.0000**	**0.0000**	**0.0000**	**0.0000**	**0.0000**	**0.0000**	**0.0000**	**0.0000**
Default Credit	0.7080	0.4372	0.6214	0.6508	0.1096	0.3534	0.0028	**0.0000**	0.0002	**0.0000**	**0.0000**	0.0010	**0.0000**	**0.0000**	**0.0000**	**0.0000**
BlogFeedback	0.6072	0.4292	0.1478	0.4744	0.3290	0.4278	0.0034	0.1084	0.0026	**0.0000**	**0.0000**	**0.0000**	**0.0000**	**0.0000**	**0.0000**	**0.0000**

Table 13
Mean monotonicity violation rates on outer-training data over five final training repetitions (lower is better).

Dataset	MLP	PWL					MixupPWL					UniformPWL				
		10^0	10^1	10^2	10^3	10^4	10^0	10^1	10^2	10^3	10^4	10^0	10^1	10^2	10^3	10^4
Abalone	0.9948	0.9912	0.9838	0.9896	0.9754	0.8498	0.6158	0.1150	0.0004	0.0002	**0.0000**	0.7084	0.1588	**0.0000**	**0.0000**	**0.0000**
Auto MPG	0.7930	0.8652	0.6256	0.5725	0.1751	0.5342	0.1412	0.0089	0.0013	**0.0000**	**0.0000**	0.0121	**0.0000**	**0.0000**	**0.0000**	**0.0000**
Boston Housing	0.0109	0.0084	0.0010	**0.0000**	**0.0000**	**0.0000**	0.0129	0.0079	0.0020	**0.0000**	**0.0000**	0.0134	0.0030	0.0010	**0.0000**	**0.0000**
COMPAS	0.6668	0.1562	0.5552	0.5012	0.3556	0.4600	0.0074	0.0004	**0.0000**	0.0002	0.0692	0.0018	0.0060	**0.0000**	**0.0000**	0.0002
ERA	0.0612	0.1577	0.0570	0.0990	0.0847	0.1600	0.0357	**0.0000**	0.0100	**0.0000**	**0.0000**	0.0280	0.0130	**0.0000**	**0.0000**	**0.0000**
ESL	0.0292	0.0108	0.0338	0.0210	0.0154	0.0128	0.0072	0.0087	0.0021	**0.0000**	**0.0000**	0.0021	**0.0000**	0.0005	**0.0000**	**0.0000**
Heart Disease	0.8595	0.2149	0.2521	0.0826	0.0810	0.0736	0.0645	**0.0050**	0.2810	0.0620	0.1479	0.0298	0.1554	0.1273	0.1471	0.1463
LEV	0.0320	0.0767	0.0255	0.0132	0.0122	**0.0000**	0.0407	0.0057	0.0062	**0.0000**	**0.0000**	0.0072	0.0045	**0.0000**	**0.0000**	**0.0000**
SWD	0.7367	0.6960	0.7045	0.6830	0.4877	0.5062	0.4095	0.0877	0.0445	0.0085	**0.0012**	0.0450	0.0190	0.0040	0.0122	0.0095
Adult	0.1954	0.0168	0.1220	0.1210	0.1550	0.0476	0.0018	0.0006	**0.0000**	0.0002	0.0032	0.0252	0.0272	0.0110	0.1966	0.1838
Default Credit	0.7388	0.7432	0.7286	0.7360	0.4086	0.2426	0.1426	0.0142	0.0024	0.0010	**0.0000**	0.6574	0.5820	0.4264	0.0080	0.3524
BlogFeedback	0.6312	0.4702	0.2508	0.2136	0.1884	0.4044	0.0318	0.0910	0.0990	**0.0004**	0.0008	0.0142	0.0166	0.0052	0.0010	0.0276

Table 14
Mean monotonicity violation rates on the fixed outer-test data over five final training repetitions (lower is better).

Dataset	MLP	PWL					MixupPWL					UniformPWL				
		10^0	10^1	10^2	10^3	10^4	10^0	10^1	10^2	10^3	10^4	10^0	10^1	10^2	10^3	10^4
Abalone	0.9935	0.9897	0.9864	0.9916	0.9715	0.8519	0.6218	0.1098	0.0007	0.0002	**0.0000**	0.7036	0.1591	**0.0000**	**0.0000**	**0.0000**
Auto MPG	0.7949	0.8684	0.6304	0.6051	0.1620	0.5519	0.1418	0.0127	**0.0000**	**0.0000**	**0.0000**	0.0177	**0.0000**	**0.0000**	**0.0000**	**0.0000**
Boston Housing	0.0079	0.0040	0.0020	**0.0000**	**0.0000**	**0.0000**	0.0099	0.0020	**0.0000**	**0.0000**	**0.0000**	0.0079	**0.0000**	**0.0000**	**0.0000**	**0.0000**
COMPAS	0.6606	0.1536	0.5476	0.4994	0.3494	0.4584	0.0092	0.0006	**0.0000**	0.0002	0.0704	0.0014	0.0044	**0.0000**	**0.0000**	0.0008
ERA	0.0610	0.1720	0.0470	0.1050	0.0720	0.1600	0.0240	**0.0000**	0.0080	**0.0000**	**0.0000**	0.0180	0.0100	**0.0000**	**0.0000**	**0.0000**
ESL	0.0184	0.0082	0.0163	0.0061	0.0082	0.0061	0.0041	0.0020	**0.0000**	**0.0000**	**0.0000**	0.0020	**0.0000**	**0.0000**	**0.0000**	**0.0000**
Heart Disease	0.8984	0.2230	0.2197	0.0820	0.0689	0.0590	0.0689	**0.0066**	0.2623	0.0623	0.1443	0.0295	0.1672	0.1180	0.1279	0.1279
LEV	0.0230	0.0760	0.0190	0.0090	0.0240	**0.0000**	0.0490	0.0020	0.0070	**0.0000**	**0.0000**	0.0040	0.0030	**0.0000**	**0.0000**	**0.0000**
SWD	0.7370	0.7080	0.7490	0.6980	0.5060	0.5280	0.4370	0.0960	0.0420	0.0120	**0.0020**	0.0530	0.0200	0.0050	0.0050	0.0070
Adult	0.2016	0.0156	0.1336	0.1166	0.1664	0.0446	0.0004	**0.0002**	**0.0002**	**0.0002**	0.0032	0.0252	0.0278	0.0122	0.2060	0.1782
Default Credit	0.7422	0.7428	0.7238	0.7318	0.3842	0.2530	0.1366	0.0168	0.0016	0.0006	**0.0000**	0.6464	0.5720	0.4280	0.0092	0.3436
BlogFeedback	0.6110	0.4574	0.2544	0.2040	0.1852	0.4230	0.0378	0.0864	0.1230	**0.0000**	0.0002	0.0136	0.0142	0.0028	0.0008	0.0264

Table 15
Pareto non-dominance rates under different monotonicity evaluation settings.

Method	Random (%)	Outer training (%)	Outer test (%)
PWL	43.33	46.67	43.33
MixupPWL	66.67	**73.33**	**75.00**
UniformPWL	**68.33**	55.00	55.00

Overall, the preferred method–λ configuration varies across datasets and evaluation regions. The best (lowest) reported primary-metric mean generally occurs at a low-to-moderate value of λ, whereas the lowest empirical violation rate is often attained at a larger value. However, neither objective changes monotonically with λ, and larger penalty coefficients can be associated with less favorable predictive or monotonicity results. PWL, MixupPWL, and UniformPWL therefore occupy different parts of the observed trade-off. In practice, the method and λ should be selected using a validation criterion that reflects the desired balance between predictive performance and monotonicity. Within the explored benchmark and regularization grid, no method or value of λ is uniformly preferable.

4.3. Discussion and practical guidelines

Based on the experimental comparison, we discuss the main empirical findings and derive practical guidelines for model selection. These observations not only clarify the comparative behavior of existing monotonicity approaches under benchmark settings but also help explain why monotonicity constraints are becoming increasingly relevant in practical neural network applications.

4.3.1. Discussion

Several instructive observations arise from the extensive experimental evaluation conducted in this work.

- Predictive performance varies substantially across the evaluated structure-based monotonicity architectures. MLP achieves the best observed average rank for predictive performance, while CoMNN, SMM, UMNN, and HLL are the four top-ranked monotonic architectures. The Nemenyi test detects no significant difference between MLP and any of these four methods. MLP_{wc} has the worst (highest) average rank and ranks last on eight datasets, although its second-best reported mean on Adult shows that it is not uniformly inferior. A simple sign-constrained baseline should therefore not be treated as representative of the broader class of monotonic architectures.
- No evaluated structure-based method is uniformly best across datasets. CoMNN achieves the best (lowest) reported primary-metric mean on Adult, Default Credit, and BlogFeedback; HLL does so on Boston Housing and ERA and matches MLP on Heart Disease at the displayed precision; UMNN does so on Auto MPG; and LMN does so on LEV. These results show that exact monotonicity guarantees within the stated scopes can coexist with competitive predictive results. However, the selected configurations differ substantially in parameter count, so the comparison should not be interpreted as an efficiency-normalized ranking.
- Auxiliary preprocessing does not provide a statistically reliable aggregate advantage in the min–max comparison. The four-method Friedman test detects no systematic difference in within-dataset ranks, and neither direct base-versus-auxiliary comparison produces a nominally significant result in the exploratory, unadjusted, two-sided Wilcoxon analysis. The dataset-level changes are also mixed in direction. Because auxiliary networks can substantially increase the selected parameter count in some configurations, their use should be evaluated on a case-by-case basis.
- Regularization-based monotonicity induction is a multi-objective tuning problem. PWL most frequently participates in the dataset-level best (lowest) reported primary-metric mean, UniformPWL has

the highest observed random-domain Pareto non-dominance rate, and MixupPWL has the highest observed outer-training and outer-test rates. The best reported predictive result is generally attained at a low-to-moderate value of λ, whereas larger values can further reduce empirical violations but may also produce less favorable trade-offs. Because the configurations are separately tuned and trained, their observed responses need not vary monotonically with λ. Within the explored benchmark and regularization grid, no method–λ configuration is uniformly preferable.

Beyond benchmarks, recent work further highlights the growing practical importance of monotonicity in neural networks. In these applications, monotonicity is not merely used as an auxiliary interpretability constraint, but increasingly serves as a principled mechanism for embedding prior knowledge, enforcing physically or clinically meaningful trends, and improving the reliability of neural predictions. In scientific and engineering modeling, monotonicity has been incorporated into physics-informed or physics-constrained neural networks for highly anisotropic diffusion equations, NO_x emission prediction in coal-fired boilers, model predictive control of building energy systems, hyperelastic constitutive modeling, and stress-dependent rock strength prediction, where monotonic constraints help encode physically consistent input–output relations, stabilize extrapolation, and improve robustness under domain-specific operating conditions (Henkel et al., 2025; Klein et al., 2026; Yao et al., 2025; Zhang & Al Kobaisi, 2022; Zhu et al., 2024). Monotonicity has also been integrated into maritime and industrial intelligence systems, for example, in privacy-preserving federated transfer learning for ship energy efficiency estimation, where monotonic priors can be incorporated into neural prediction pipelines under realistic data-sharing constraints (Wang et al., 2025b). In healthcare decision support, domain constraints should be accompanied by reliable knowledge representation and transparent explanations so that model outputs remain clinically meaningful and auditable (Domingues, 2026; Gribova & Shalfeeva, 2025). Consistent with these requirements, monotonic neural models have been explored for competing-risks survival analysis, general survival regression, first-hitting time prediction, and Alzheimer's disease progression modeling, where monotone temporal or risk-related trends align naturally with clinically meaningful progression patterns and improve the trustworthiness of longitudinal prediction (Chen et al., 2025; Jeanselme et al., 2023; Jeong et al., 2025; Rindt et al., 2022). In finance and regulated decision-making, monotonic neural additive models have been proposed for credit scoring, and related monotonic neural formulations have also been explored in health-risk pricing, reflecting the need for neural models whose outputs remain consistent with domain expectations, interpretability requirements, and regulatory constraints (Chen & Ye, 2022). Related monotonic ordering constraints have also proved useful in probabilistic forecasting and structured prediction, including monotone quantile regression for time-series uncertainty estimation, learning probability distributions of day-ahead electricity prices, and deep lattice network formulations for nonparametric cumulative distribution forecasting, where monotonicity is essential for ensuring valid neural outputs and avoiding quantile crossing (Erdmann et al., 2026; Hanus & Baruník, 2025; Hu et al., 2024). More broadly, monotonic mechanisms have also been introduced into tabular representation learning, suggesting that monotonicity is increasingly used not only as an interpretability tool but also as a practically meaningful inductive bias in neural networks to improve reliability, stability, and trustworthy deployment across diverse domains (Wu et al., 2026). Recent studies further suggest that this inductive bias is beginning to extend beyond traditional tabular prediction and structured decision settings into foundation models and safety-critical dynamical systems. For example, monotonicity has been introduced as an architectural bias in Transformer-based language models to improve robustness against adversarial and jailbreak attacks (Cooper et al., 2026), while related monotone neural constructions have also been explored for barrier-certificate learning in high-dimensional monotone dynamical systems, highlighting the emerging role of monotonicity in scalable safety verification and control synthesis (Jafarpour et al., 2026).

In summary, these studies indicate that monotonicity constraints are increasingly viewed as a practical requirement rather than merely a theoretical modeling preference, driven by factors such as physical laws, safety considerations, regulatory requirements, and decision consistency. Meanwhile, most application-oriented research relies on task-specific formulations without explicitly examining broader issues such as monotonicity guarantees, expressiveness trade-offs, and performance comparisons across different methods. This gap underscores the need to establish a systematic taxonomy and a unified empirical evaluation, which motivates this survey. Overall, existing experiments and emerging applications suggest that the design of monotonicity constraints in neural networks should be treated as a multi-objective problem balancing prediction performance, monotonicity guarantees, model complexity, and optimization stability. Since no evaluated method dominates across all benchmark datasets and evaluation criteria, informed model selection remains inherently application-dependent.

4.3.2. Practical guidelines

Beyond methodological comparisons, this paper's experimental results offer practical insights into selecting different monotonicity constraints for neural networks in real-world applications.

- When a domain-wide monotonicity guarantee is required, and the architectural assumptions match the application, exact structure-based approaches provide the most direct solution. CoMNN, SMM, UMNN, and HLL achieve comparatively favorable average predictive-performance ranks in this benchmark while guaranteeing monotonicity by construction within their stated scopes. Selection among them should additionally consider input dimensionality, optimization requirements, and parameter count.
- When architectural flexibility is the primary consideration and empirical monotonicity induction is sufficient, PWL, MixupPWL, and UniformPWL provide tunable alternatives. PWL most frequently participates in the dataset-level best (lowest) reported primary-metric mean, UniformPWL has the highest observed random-domain Pareto non-dominance frequency, and MixupPWL has the highest observed outer-training and outer-test Pareto non-dominance frequencies. In practice, the method–λ configuration should be selected using a validation criterion that reflects the desired predictive–monotonicity balance and the intended deployment region.
- Parameter efficiency should be assessed from the selected configuration. UMNN and SMNN have the largest median parameter counts in this benchmark, whereas MM, SMM, LMN, CoMNN, and MLP_{wc} are generally more compact. Nevertheless, the smallest configuration varies across datasets, and parameter count alone does not explain predictive rank.
- Random-domain, outer-training, and outer-test violation rates should be matched to the application's relevant operating region and reported separately. The experiments show that the three finite-sample audit rates can differ substantially; none should be interpreted as a formal domain-wide monotonicity guarantee.

Overall, the experimental results suggest that effective monotonicity design depends on matching the enforcement mechanism to the application context, the required guarantee strength, and the acceptable level of model complexity.

5. Challenges and future directions

Despite recent progress, several theoretical and practical challenges remain in incorporating monotonicity constraints into neural networks. This section summarizes the main open challenges and outlines promising future directions.

Table 16
Summary of challenges and corresponding research directions.

Challenge	Key issues	Corresponding research direction
Trade-off between monotonicity guarantees and model expressiveness	Strong architectural constraints may reduce model flexibility and limit representational capacity, while simple parameter restrictions may yield less favorable predictive performance in complex learning tasks.	Develop more expressive monotonic architectures that preserve their stated guarantees while retaining greater modeling flexibility and competitive predictive performance.
Limited scalability of architectures with by-construction monotonicity guarantees	Some architectures with exact monotonicity guarantees rely on specialized structural components whose computational or representational cost may grow rapidly with input dimensionality or model complexity.	Design scalable monotonic architectures, including efficient modular designs and partially monotonic components suitable for high-dimensional learning.
Difficulty of balancing predictive performance and monotonicity induction	Regularization-based strategies are sensitive to hyperparameter choices; overly strong penalties may degrade predictive performance, whereas overly weak penalties may leave substantial empirical monotonicity violations.	Develop adaptive and validation-aware training strategies for balancing predictive performance and monotonicity, such as dynamic regularization schedules and multi-objective optimization.
Computational cost of formal verification	Formal certification methods can provide guarantees for specified models, domains, and assumptions, but their computational cost may become substantial for deep networks or high-dimensional domains.	Develop efficient and scalable certification methods, including tighter sound relaxations, sound but incomplete verification procedures, and hybrid training–verification pipelines.
Insufficient alignment between monotonicity design and application requirements	Generic monotonic constraints may not adequately reflect domain-specific requirements concerning which features should be monotonic, where the relationships should hold, and what guarantee strength is required.	Promote application-driven monotonicity design in which constraint directions, domains, scopes, and guarantee strengths are determined using domain knowledge and deployment requirements.

5.1. Challenges

Despite the significant progress in incorporating monotonicity constraints into neural networks, several challenges remain.

- **Trade-off between monotonicity guarantees and model expressiveness.** Many structure-based approaches provide monotonicity guarantees by construction within their stated scopes and assumptions, using architectural constraints such as non-negative weights and monotonic activation functions. Although these mechanisms provide direct guarantees, they may reduce model flexibility and restrict the functions that can be represented efficiently. The comparatively weak predictive performance of $\mathrm{MLP}_{\mathrm{wc}}$ in our experiments is consistent with reduced expressive flexibility under simple sign constraints, although the present evaluation does not directly establish underfitting. Preserving expressive capacity while retaining by-construction monotonicity guarantees therefore remains an important challenge.
- **Limited scalability of architectures with by-construction monotonicity guarantees.** Some architectures that provide exact monotonicity guarantees rely on specialized components, such as lattice layers and constrained transformations, whose computational or representational cost may grow rapidly with input dimensionality or model complexity. Designing scalable architectures that retain stated monotonicity guarantees without imposing high computational cost remains challenging.
- **Difficulty of balancing predictive performance and monotonicity induction.** Training-based monotonicity induction mechanisms typically introduce penalty terms for observed monotonicity violations. Selecting an appropriate penalty strength is difficult: very large coefficients may reduce predictive performance, whereas small coefficients may leave substantial empirical violations. Our results show that the observed trade-off and preferred configuration vary across datasets and evaluation regions. Although adaptive formulations exist, developing robust, validation-aware mechanisms to calibrate monotonicity strength across tasks remains an open problem.
- **Computational cost of formal verification.** Verification-based mechanisms can provide certificates using MILP, SMT/OMT, or Lipschitz–Voronoi coverage, but they certify different objects and rely on different model, domain, and smoothness assumptions. Their computational cost can increase substantially for deep networks or high-dimensional domains, making scalable certification a key challenge for practical deployment.
- **Insufficient alignment between monotonicity design and application requirements.** Many existing formulations prespecify monotonic directions and scopes without fully representing application-specific requirements. Different domains may require different constrained features, monotonic directions, valid regions, and guarantee strengths. Translating these requirements into principled, domain-informed, data-aware, and computationally feasible neural network designs remains an important open challenge.

5.2. Future directions

Despite the effectiveness of the existing methods, there are still many future directions that require further investigation.

- **Expressive monotonic architectures with stronger generalization ability.** The trade-off between monotonicity guarantees and model expressiveness remains a central challenge. Future research should develop architectures that provide direct monotonicity guarantees within clearly stated scopes and assumptions while retaining sufficient modeling flexibility. Promising directions include more expressive partially monotonic architectures, hybrid monotonic–unconstrained designs, and parameterizations that preserve monotonicity without excessively restricting the hypothesis space.
- **Scalable monotonic architectures and scope-aware enforcement.** Some architectures with by-construction monotonicity guarantees rely on specialized components whose cost can increase rapidly with input dimensionality or model complexity. Future work should develop more computationally efficient architectures that preserve their stated guarantees in high-dimensional settings. When monotonicity is required only over specified features or regions, promising directions include modular monotonic components, explicitly defined region-specific constraints, and architectures that accommodate different constraint scopes across feature groups.
- **Adaptive training strategies for balancing predictive performance and monotonicity.** The dataset- and region-dependent responses observed across the explored regularization strengths motivate more adaptive approaches to balancing predictive performance and empirical monotonicity. Although adaptive formulations already exist, further research is needed to develop robust and validation-aware strategies, including dynamic penalty adjustment, constrained or multi-objective optimization, and selection criteria that reflect the monotonicity requirements of the intended deployment region.

- **Efficient and scalable certification of monotonicity.** Formal verification methods can provide certificates for specified models, domains, and assumptions, but their computational cost can become substantial for deep networks or high-dimensional domains. Future research should improve the efficiency and scalability of monotonicity certification through tighter sound relaxations, sound but incomplete or anytime verification procedures, compositional certification, and hybrid training–verification pipelines that facilitate subsequent certification.
- **Application-driven monotonicity design.** As monotonicity constraints are increasingly considered in safety-critical and highly regulated domains, future research should investigate how to translate application requirements into principled monotonicity specifications. Domain knowledge should guide the selection of constrained features, monotonic directions, valid regions, and required guarantee strength. Bridging general-purpose monotonic methods with application-specific requirements remains particularly important in healthcare, finance, and decision-support systems.

5.3. Summary of challenges and corresponding research directions.

The challenges and future directions discussed above highlight that incorporating monotonicity constraints into neural networks remains a complex and evolving research topic. Existing methods differ substantially in their design philosophies, ranging from architecture-level integration to optimization-based enforcement and post-training verification. Each paradigm offers different trade-offs among monotonicity guarantees, expressive power, computational efficiency, and implementation complexity. Table 16 summarizes the main challenges identified in this survey and the corresponding research directions that may address them.

Overall, monotonicity constraints in neural networks should not be viewed as a single modeling paradigm, but rather as a flexible design framework that integrates domain knowledge into learning systems. Future research is expected to further explore how monotonic constraints can be incorporated in a principled, scalable, and application-driven manner to support trustworthy and interpretable machine learning models.

6. Conclusion

This paper systematically reviews monotonicity constraints in neural networks and empirically evaluates representative approaches. We propose a unified taxonomy that divides the reviewed methods into structure-based and non-structural monotonicity approaches and elucidates their design principles and trade-offs among monotonicity guarantees, model flexibility, and optimization complexity.

Based on this taxonomy, we conduct systematic empirical evaluations on 12 benchmark datasets. MLP obtains the best average predictive rank. Among the monotonic architectures, CoMNN, SMM, UMNN, and HLL obtain the four best observed average ranks, and the multiplicity-controlled Nemenyi test detects no significant difference between MLP and any of these four methods. MLP_{wc} has the weakest average rank, indicating that the performance of a simple sign-constrained baseline should not be treated as representative of monotonic architectures more broadly. Dataset-level leaders and selected parameter counts vary substantially, so these results compare complete, independently tuned configurations rather than establish parameter-normalized architectural superiority. In the regularization study, PWL most often achieves the lowest reported predictive mean among the three regularized methods when ties at the displayed precision are retained. UniformPWL has the highest observed random-domain Pareto non-dominance rate, whereas MixupPWL has the highest observed outer-training and outer-test rates. No evaluated method–λ configuration is uniformly preferable across datasets, objectives, and evaluation regions.

Overall, this survey establishes a unified perspective for understanding and comparing monotonicity constraints in neural networks while highlighting the importance of selecting monotonicity mechanisms according to application requirements, data characteristics, guarantee strength, and performance objectives. Monotonic neural networks should be assessed jointly in terms of predictive performance, monotonicity guarantees, model complexity, and optimization requirements rather than solely by empirical monotonicity satisfaction. The empirical benchmark spans 12 real-world tabular datasets, ranging from classical monotonic-learning benchmarks to larger and higher-dimensional tasks, with predictive performance evaluated on fixed held-out test partitions. Future work should extend this framework to more complex application domains and develop efficient monotonic models that retain clearly specified guarantees as input dimensionality and model complexity increase.

CRediT authorship contribution statement

Junpeng Sun: Writing – original draft, Software, Methodology, Data curation; **Juan Luis Suárez:** Writing – review & editing, Visualization, Investigation, Formal analysis; **Weiping Ding:** Writing – review & editing, Validation, Supervision, Funding acquisition; **Salvador García:** Writing – review & editing, Supervision, Methodology, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work is supported in part by the China Scholarship Council (No. 202409040011), the project TSI-100927-2023-1, funded by the European Union's NextGenerationEU programme under the Recovery, Transformation and Resilience Plan, through the Ministry for Digital Transformation and Public Administration, the National Key R&D Plan of China under Grant No. 2024YFE0202700, the National Natural Science Foundation of China under Grant Nos. 62576178 and U2433216, and the Spanish Ministry of Science and Technology under project PID2023-150070NB-I00, including European Regional Development Funds.

References

Abdi, H., Valentin, D., & Edelman, B. (1999). Neural networks. Sage.

Abu-Mostafa, Y. S. (1990). Learning from hints in neural networks. *Journal of Complexity*, *6*(2), 192–198.

Alon, N., & Boppana, R. B. (1987). The monotone circuit complexity of boolean functions. *Combinatorica*, *7*(1), 1–22.

Amos, B., Xu, L., & Kolter, J. Z. (2017). Input convex neural networks. In *Proceedings of the 34th international conference on machine learning* (pp. 146–155).

Angwin, J., Larson, J., Mattu, S., & Kirchner, L. (2016). Machine bias. ProPublica.

Anil, C., Lucas, J., & Grosse, R. (2019). Sorting out lipschitz function approximation. In *Proceedings of the international conference on machine learning* (pp. 291–301).

Archer, N. P., & Wang, S. (1993). Application of the back propagation neural network algorithm with monotonicity constraints for two-group classification problems. *Decision Sciences*, *24*(1), 60–75.

Bai, Y., Zhang, S., Zhang, Y., Liu, H., Bao, W., Yu, E., Feng, F., & Ou, W. (2025). Unconstrained monotonic calibration of predictions in deep ranking systems. In *Proceedings of the 48th international ACM SIGIR conference on research and development in information retrieval* (pp. 937–947).

Bakst, W. T., Morioka, N., & Louidor, E. (2021). Monotonic kronecker-factored lattice. In *International conference on learning representations*.

Banerjee, C., Mukherjee, T., & Pasiliao Jr , E., (2019). An empirical study on generalizations of the reLU activation function. In *Proceedings of the 2019 ACM southeast conference* (pp. 164–167).

Barredo Arrieta, A., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., Garcia, S., Gil-Lopez, S., Molina, D., Benjamins, R., Chatila, R., & Herrera, F. (2020). Explainable artificial intelligence (XAI): concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion*, *58*, 82–115.

Bartley, C., Liu, W., & Reynolds, M. (2018). A novel framework for constructing partially monotone rule ensembles. In *2018 IEEE 34th international conference on data engineering* (pp. 1320–1323).

Bartley, C., Liu, W., & Reynolds, M. (2019). Enhanced random forest algorithms for partially monotone ordinal classification. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 3224–3231).

Becker, B., & Kohavi, R. (1996). Adult. UCI Machine Learning Repository.

Ben-David, A., Sterling, L., & Pao, Y.-H. (1989). Learning and classification of monotonic ordinal concepts. *Computational Intelligence*, *5*(1), 45–49.

Bunel, R. R., Turkaslan, I., Torr, P., Kohli, P., & Mudigonda, P. K. (2018). A unified view of piecewise linear neural network verification. In *Advances in neural information processing systems*.

Buza, K. (2014). BlogFeedback. UCI Machine Learning Repository.

Cano, J.-R., & García, S. (2017). Training set selection for monotonic ordinal classification. *Data & Knowledge Engineering*, *112*, 94–105.

Capstick, A., Palermo, F., Cui, T., & Barnaghi, P. (2025). Training neural networks on data sources with unknown reliability. *Information Fusion*, *123*, 103239.

Chatterjee, S., & Mukherjee, S. (2019). Estimation in tournaments and graphs under monotonicity constraints. *IEEE Transactions on Information Theory*, *65*(6), 3525–3539.

Chen, A.-S., Leung, M. T., & Daouk, H. (2003). Application of neural networks to an emerging financial market: Forecasting and trading the taiwan stock index. *Computers & Operations Research*, *30*(6), 901–923.

Chen, D., & Ye, W. (2022). Monotonic neural additive models: Pursuing regulated machine learning models for credit scoring. In *Proceedings of the ACM international conference on AI in finance* (pp. 70–78).

Chen, E.-C., Chen, P.-Y., Chung, I., Lee, C.-R. et al. (2026). Data-driven lipschitz continuity: A cost-effective approach to improve adversarial robustness. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision* (pp. 698–707).

Chen, Y. K., Dittmer, S., Bernatowicz, K., Arús-Pous, J., Bliznashki, K., Aston, J., Rudd, J. H. F., Schönlieb, C.-B., Jones, J., & Roberts, M. (2025). SurvSurf: a partially monotonic neural network for first-hitting time prediction of intermittently observed discrete and continuous sequential events. arXivpreprint arXiv:2504.04997

Cheng, H., Yang, Y., Tao, H., & Zhang, F. (2026). Isotonic layer: A universal framework for generic recommendation debiasing. arXivpreprint arXiv:2603.06589

Chuang, H.-C., Chen, C.-C., & Li, S.-T. (2024). Advancing SVM classification: Parallelizing conjugate gradient for monotonicity enforcement. *Knowledge-Based Systems*, *302*, 112388.

Cooper, P., Nadali, A., Trivedi, A., & Velasquez, A. (2026). Monotonicity as an architectural bias for robust language models. arXivpreprint arXiv:2602.02686

Cotter, A., Gupta, M., Jiang, H., Louidor, E., Muller, J., Narayan, T., Wang, S., & Zhu, T. (2019). Shape constraints for set functions. In *Proceedings of the international conference on machine learning* (pp. 1388–1396).

Daniels, H., & Velikova, M. (2010). Monotone and partially monotone neural networks. *IEEE Transactions on Neural Networks*, *21*(6), 906–917.

Dash, T., Chitlangia, S., Ahuja, A., & Srinivasan, A. (2022). A review of some techniques for inclusion of domain-knowledge into deep neural networks. *Scientific Reports*, *12*(1), 1040.

Defazio, A., Yang, X., Mehta, H., Mishchenko, K., Khaled, A., & Cutkosky, A. (2024). The road less scheduled. In *Advances in neural information processing systems* (pp. 9974–10007).

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, *7*, 1–30.

Ding, W., Ming, Y., Cao, Z., & Lin, C.-T. (2022). A generalized deep neural network approach for digital watermarking analysis. *IEEE Transactions on Emerging Topics in Computational Intelligence*, *6*(3), 613–627.

Domingues, N. S. (2026). From engineering principles to healthcare practice: A hybrid reasoning framework for transparent clinical decision support. *Artificial Intelligence in Health*, *3*(2), 025470102.

Dugas, C., Bengio, Y., Bélisle, F., Nadeau, C., & Garcia, R. (2009). Incorporating functional knowledge in neural networks. *Journal of Machine Learning Research*, *10*, 1239–1262.

Erdmann, N., Bentsen, L., Stenbro, R., Riise, H. N., Warakagoda, N. D., & Engelstad, P. E. (2026). Multi-horizon time series forecasting of non-parametric CDFs with deep lattice networks. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 255–264).

Fiori, S., Gong, T., & Lee, H. K. (2015). Bivariate nonisotonic statistical regression by a lookup table neural system. *Cognitive Computation*, *7*(6), 715–730.

Friedman, M. (1937). The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association*, *32*(200), 675–701.

Gagneux, A., Massias, M., Soubies, E., & Gribonval, R. (2025). Convexity in reLU neural networks: Beyond ICNNs? *Journal of Mathematical Imaging and Vision*, *67*, 40.

Ghugare, R., & Eysenbach, B. (2025). Normalizing flows are capable models for continuous control. In *Advances in neural information processing systems* (pp. 78796–78823).

Gomez-Rubio, V. (2018). Generalized additive models: An introduction with r. *Journal of statistical software*, *86*, 1–5.

González-Almagro, G., Sánchez-Bermejo, P., Suarez, J. L., Cano, J.-R., & García, S. (2024). Semi-supervised clustering with two types of background knowledge: Fusing pairwise constraints and monotonicity constraints. *Information Fusion*, *102*, 102064.

Gouk, H., Frank, E., Pfahringer, B., & Cree, M. J. (2021). Regularisation of neural networks by enforcing lipschitz continuity. *Machine Learning*, *110*(2), 393–416.

Goyal, K., Dumancic, S., & Blockeel, H. (2024). Deepsade: Learning neural networks that guarantee domain constraint satisfaction. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 12199–12207).

Gribova, V., & Shalfeeva, E. (2025). Explainable solutions from artificial intelligence for health-care support systems. *Artificial Intelligence in Health*, *2*(3), 138–153.

Gu, J., Shen, Y., Chen, T., Dinh, L., Wang, Y., Bautista, M. A., Berthelot, D., Susskind, J., & Zhai, S. (2025). Starflow-v: End-to-end video generative modeling with normalizing flows. arXivpreprint arXiv:2511.20462

Gupta, A., Shukla, N., Marla, L., Kolbeinsson, A., & Yellepeddi, K. (2019). How to incorporate monotonicity in deep networks while preserving flexibility? arXivpreprint arXiv:1909.10662

Gupta, M., Bahri, D., Cotter, A., & Canini, K. (2018). Diminishing returns shape constraints for interpretability and regularization. In *Advances in neural information processing systems*.

Gupta, M., Cotter, A., Pfeifer, J., Voevodski, K., Canini, K., Mangylov, A., Moczydlowski, W., & Van Esbroeck, A. (2016). Monotonic calibrated interpolated look-up tables. *Journal of Machine Learning Research*, *17*(109), 1–47.

Hanus, L., & Baruník, J. (2025). Learning the probability distributions of day-ahead electricity prices. *Energy Economics*, *152*, 108988.

Harrison, D., & Rubinfeld, D. L. (1978). Hedonic housing prices and the demand for clean air. *Journal of Environmental Economics and Management*, *5*(1), 81–102.

Henkel, P., Roß, S., Rätz, M., & Müller, D. (2025). Monotonic physics-constrained neural networks for model predictive control of building energy systems. *Building and Environment*, *285*, 113640.

Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, *2*(5), 359–366.

Hu, J., Tang, J., & Liu, Z. (2024). A novel time series probabilistic prediction approach based on the monotone quantile regression neural network. *Information Sciences*, *654*, 119844.

Huang, C.-W., Krueger, D., Lacoste, A., & Courville, A. (2018). Neural autoregressive flows. In *Proceedings of the international conference on machine learning* (pp. 2078–2087).

Huang, G.-B., Zhu, Q.-Y., & Siew, C.-K. (2006). Extreme learning machine: Theory and applications. *Neurocomputing*, *70*(1), 489–501.

Hurault, A., & Marques-Silva, J. (2023). Certified logic-based explainable AI–the case of monotonic classifiers. In *International conference on tests and proofs* (pp. 51–67).

Huster, T., Chiang, C.-Y. J., & Chadha, R. (2018). Limitations of the lipschitz constant as a defense against adversarial examples. In *Proceedings of the joint european conference on machine learning and knowledge discovery in databases* (pp. 16–29).

Hušek, P. (2018). Monotonicity conditions for radial basis function networks. In *Proceedings of the IEEE symposium series on computational intelligence* (pp. 168–173).

Hušek, P. (2024). On monotonic radial basis function networks. *IEEE Transactions on Cybernetics*, *54*(2), 717–727.

Igel, C. (2024). Smooth min-max monotonic networks. In *Proceedings of the international conference on machine learning* (pp. 20908–20923).

Jadoon, A. A., Seidl, D. T., Jones, R. E., & Fuhg, J. N. (2026). Input specific neural networks. *Computational Mechanics*, *77*, 1959–1981.

Jafarpour, S., Nadali, A., Trivedi, A., & Zamani, M. (2026). Monotone neural barrier certificates. arXivpreprint arXiv:2508.12178

Janosi, A., Steinbrunn, W., Pfisterer, M., & Detrano, R. (1988). Heart Disease. UCI Machine Learning Repository.

Jazbec, M., Allingham, J., Zhang, D., & Nalisnick, E. (2023). Towards anytime classification in early-exit architectures by enforcing conditional monotonicity. In *Advances in neural information processing systems* (pp. 56138–56168).

Jeanselme, V., Yoon, C. H., Tom, B., & Barrett, J. (2023). Neural fine-gray: Monotonic neural networks for competing risks. In *Proceedings of the conference on health, inference, and learning* (pp. 379–392).

Jeong, S., Jung, W., Sohn, J., & Suk, H.-I. (2025). Deep geometric learning with monotonicity constraints for alzheimer's disease progression. *IEEE Transactions on Neural Networks and Learning Systems*, *36*(4), 7090–7102.

Kalbfleisch, J. D. (1975). Statistical inference under order restrictions. *Technometrics*, *17*(1), 139–140.

Kaplan, H., Weichselbraun, A., & Braşoveanu, A. M. P. (2023). Integrating economic theory, domain knowledge, and social knowledge into hybrid sentiment models for predicting crude oil markets. *Cognitive Computation*, *15*(4), 1355–1371.

Kasongo, S. M. (2023). A deep learning technique for intrusion detection system using a recurrent neural networks based framework. *Computer Communications*, *199*, 113–125.

Katz, G., Barrett, C., Dill, D. L., Julian, K., & Kochenderfer, M. J. (2017). Reluplex: An efficient SMT solver for verifying deep neural networks. In *International conference on computer aided verification* (pp. 97–117).

Khalaf, O., Garcia, S., & Ben Ishak, A. (2025). Generalised entropies for decision trees in classification under monotonicity constraints. *Expert Systems*, *42*, e70035.

Khalaf, O., Ishak, A. B., & García, S. (2026). Towards explainable multilabel learning: Fusing label dependency analysis with monotonic decision trees. *Information Fusion*, *127*, 103691.

Khromov, G., & Singh, S. P. (2024). Some fundamental aspects about lipschitz continuity of neural networks. In *International conference on learning representations* (pp. 4261-4305).

Kim, H., & Lee, J.-S. (2024). Scalable monotonic neural networks. In *Proceedings of the international conference on learning representations*.

Kingma, D. P., Salimans, T., Jozefowicz, R., Chen, X., Sutskever, I., & Welling, M. (2016). Improved variational inference with inverse autoregressive flow. In *Advances in neural information processing systems*.

Klein, D. K., Hossain, M., Kikinov, K., Kannapinn, M., Rudykh, S., & Gil, A. J. (2026). Neural networks meet hyperelasticity: A monotonic approach. *European Journal of Mechanics - A/Solids*, *116*, 105900.

Kobyzev, I., Prince, S. J. D., & Brubaker, M. A. (2021). Normalizing flows: An introduction and review of current methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *43*(11), 3964–3979.

Kotlowski, W., & Slowinski, R. (2013). On nonparametric ordinal classification with monotonicity constraints. *IEEE Transactions on Knowledge and Data Engineering*, *25*(11), 2576–2589.

Kumar Jain, D., Zhao, X., Garcia, S., & Neelakandan, S. (2024). Robust multi-modal pedestrian detection using deep convolutional neural network with ensemble learning model. *Expert Systems with Applications*, *249*, 123527.

Kyng, R., Rao, A., & Sachdeva, S. (2015). Fast, provable algorithms for isotonic regression in all l_p-norms. In *Advances in neural information processing systems*.

Leitenstorfer, F., & Tutz, G. (2007). A boosting approach to generalized monotonic regression. In *Advances in data analysis: Proceedings of the annual conference of the gesellschaft für klassifikation e.v.* (pp. 245–254).

Levy, S., Laloy, E., & Linde, N. (2023). Variational bayesian inference with complex geostatistical priors using inverse autoregressive flows. *Computers & Geosciences*, *171*, 105263.

Li, H., Chen, J., Wang, B., García, S., & Camacho, D. (2026). Explainable artificial intelligence for deepfake detection: Pipeline, open source and comparisons. *Expert Systems*, *43*(3), e70222.

Li, S.-T., & Chen, C.-C. (2015). A regularized monotonic fuzzy support vector machine model for data mining with prior knowledge. *IEEE Transactions on Fuzzy Systems*, *23*(5), 1713–1727.

Li, X., Sun, L., Ling, M., & Peng, Y. (2023). A survey of graph neural network based recommendation in social networks. *Neurocomputing*, *549*, 126441.

Liew, S. S., Khalil-Hani, M., & Bakhteri, R. (2016). Bounded activation functions for enhanced training stability of deep neural networks on visual pattern recognition problems. *Neurocomputing*, *216*, 718–734.

Liu, H., Luo, W., Han, L., Gao, P., Yang, W., & Han, G. (2024). Anomaly detection via graph attention networks-augmented mask autoregressive flow for multivariate time series. *IEEE Internet of Things Journal*, *11*(11), 19368–19379.

Liu, X., Han, X., Zhang, N., & Liu, Q. (2020). Certified monotonic neural networks. In *Advances in neural information processing systems* (pp. 15427–15438).

Liu, Z., Wang, Y., Vaidya, S., Ruehle, F., Halverson, J., Soljacic, M., Hou, T., & Tegmark, M. (2025). Kan: Kolmogorov–arnold networks. In *International conference on learning representations*.

Lowe, D., & Broomhead, D. (1988). Multivariable functional interpolation and adaptive networks. *Complex Systems*, *2*(3), 321–355.

Majerz, E., Dzwinel, W., & Kitowski, J. (2025). Inverse autoregressive flows for zero degree calorimeter fast simulation. arXivpreprint arXiv:2512.20346

Mari, M., & Snidaro, L. (2026). Survey of neural network approaches to target tracking with an emphasis on interpretability. *Information Fusion*, *127*, 103789.

Mikulincer, D., & Reichman, D. (2025). Size and depth of monotone neural networks: interpolation and approximation. *IEEE Transactions on Neural Networks and Learning Systems*, *36*(4), 6314–6325.

Milani Fard, M., Canini, K., Cotter, A., Pfeifer, J., & Gupta, M. (2016). Fast and flexible monotonic functions with ensembles of lattices. In *Advances in neural information processing systems*.

Monteiro, J., Ahmed, M. O., Hajimirsadeghi, H., & Mori, G. (2021). Not too close and not too far: Enforcing monotonicity requires penalizing the right points. In *Proceedings of the explainable artificial intelligence for debugging and diagnosis workshop*.

Monteiro, J., Ahmed, M. O., Hajimirsadeghi, H., & Mori, G. (2022). Monotonicity regularization: Improved penalties and novel applications to disentangled representation learning and robust classification. In *Proceedings of the conference on uncertainty in artificial intelligence* (pp. 1381–1391).

Motie, S., & Raahemi, B. (2024). Financial fraud detection using graph neural networks: A systematic review. *Expert Systems with Applications*, *240*, 122156.

Muralidhar, N., Islam, M. R., Marwah, M., Karpatne, A., & Ramakrishnan, N. (2018). Incorporating prior domain knowledge into deep neural networks. In *Ieee international conference on big data* (pp. 36–45).

Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted boltzmann machines. In *Proceedings of the international conference on machine learning* (pp. 807–814).

Nash, W., Sellers, T., Talbot, S., Cawthorn, A., & Ford, W. (1995). Abalone. UCI Machine Learning Repository.

Nemenyi, P. B. (1963). Distribution-free multiple comparisons. Princeton University.

Nguyen, A.-P., Moreno, D. L., Le-Bel, N., & Rodríguez Martínez, M. (2023). MonoNet: Enhancing interpretability in neural networks via monotonic features. *Bioinformatics Advances*, *3*(1), vbad016.

Nolte, N., Kitouni, O., & Williams, M. (2023). Expressive monotonic neural networks. In *The eleventh international conference on learning representations*.

Papamakarios, G., Pavlakou, T., & Murray, I. (2017). Masked autoregressive flow for density estimation. In *Advances in neural information processing systems*.

Pawlicki, M., Kozik, R., & Choraś, M. (2022). A survey on neural networks for (cyber-) security and (cyber-) security of neural networks. *Neurocomputing*, *500*, 1075–1087.

Petersen, B. K., Santiago, C., & Landajuela, M. (2021). Incorporating domain knowledge into neural-guided search via in situ priors and constraints. In *8th ICML workshop on automated machine learning*.

Polo-Molina, A., Alfaya, D., & Portela, J. (2026a). A mathematical certification for positivity conditions in neural networks with applications to partial monotonicity and trustworthy AI. *IEEE Transactions on Neural Networks and Learning Systems*, *37*(2), 981–996.

Polo-Molina, A., Alfaya, D., & Portela, J. (2026b). MonoKAN: Certified monotonic kolmogorov-arnold network. *Neural Networks*, *196*, 108278.

Potharst, R., & Feelders, A. J. (2002). Classification trees for problems with monotonicity constraints. *ACM SIGKDD Explorations Newsletter*, *4*(1), 1–10.

Quinlan, J. R. (1993). Auto MPG. UCI Machine Learning Repository.

Ramsay, J. O. (1988). Monotone regression splines in action. *Statistical Science*, *3*(4), 425–441.

Razborov, A. A. (1985). Lower bounds on the monotone complexity of some boolean functions. *Soviet Mathematics Doklady*, *31*, 354–357.

Rindt, D., Hu, R., Steinsaltz, D., & Sejdinovic, D. (2022). Survival regression with proper scoring rules and monotonic neural networks. In *Proceedings of the international conference on artificial intelligence and statistics* (pp. 1190–1205).

Rodriguez, D., Valdora, M., & Vena, P. (2022). Robust estimation in partially linear regression models with monotonicity constraints. *Communications in Statistics - Simulation and Computation*, *51*(4), 2039–2052.

Rössig, A., & Petkovic, M. (2021). Advances in verification of reLU neural networks. *Journal of Global Optimization*, *81*(1), 109–152.

Roychowdhury, S., Diligenti, M., & Gori, M. (2021). Regularizing deep networks with prior knowledge: A constraint-based approach. *Knowledge-Based Systems*, *222*, 106989.

Runje, D., & Shankaranarayana, S. M. (2023). Constrained monotonic neural networks. In *Proceedings of the international conference on machine learning* (pp. 29338–29353).

Salim, J., Yu, J., & Zhao, X. (2025). Dim: Enforcing domain-informed monotonicity in deep neural networks. arXivpreprint arXiv:2509.21666

Sartor, D., Sinigaglia, A., & Susto, G. A. (2025). Advancing constrained monotonic neural networks: achieving universal approximation beyond bounded activations. In *Proceedings of the international conference on machine learning* (pp. 52995–53018).

Scheidt, C., Mathieu, L., Yin, Z., Wang, L., & Caers, J. (2025). Masked autoregressive flow for geochemical anomaly detection with application to li–cs–ta pegmatites exploration of the superior craton, canada. *Natural Resources Research*, *34*(1), 1–22.

Shang, W., Sohn, K., Almeida, D., & Lee, H. (2016). Understanding and improving convolutional neural networks via concatenated rectified linear units. In *Proceedings of the international conference on machine learning* (pp. 2217–2225).

Sill, J. (1997). Monotonic networks. In *Advances in neural information processing systems*.

Sill, J., & Abu-Mostafa, Y. (1996). Monotonicity hints. In *Advances in neural information processing systems*.

Sill, J., & Abu-Mostafa, Y. (1997). Monotonicity: Theory and implementation. In *Intelligent methods in signal processing and communications* (pp. 129–146).

Silva, W., Fernandes, K., Cardoso, M. J., & Cardoso, J. S. (2018). Towards complementary explanations using deep neural networks. In *Understanding and interpreting machine learning in medical image computing applications* (pp. 133–140).

Silvestri, M., Lombardi, M., & Milano, M. (2021). Injecting domain knowledge in neural networks: A controlled experiment on a constrained problem. In *Integration of constraint programming, artificial intelligence, and operations research* (pp. 266–282).

Sivaraman, A., Farnadi, G., Millstein, T., & Van den Broeck, G. (2020). Counterexample-guided learning of monotonic neural networks. In *Advances in neural information processing systems* (pp. 11936–11948).

Stewart, R., & Ermon, S. (2017). Label-free supervision of neural networks with physics and domain knowledge. In *Proceedings of the AAAI conference on artificial intelligence*.

Suárez, J. L., González-Almagro, G., García, S., & Herrera, F. (2024). Metric learning for monotonic classification: Turning the space up to the limits of monotonicity. *Applied Intelligence*, *54*(5), 4443–4466.

Tang, C., Ji, J., Todo, Y., Shimada, A., Ding, W., & Hirata, A. (2024). Dendritic neural network: A novel extension of dendritic neuron model. *IEEE Transactions on Emerging Topics in Computational Intelligence*, *8*(3), 2228–2239.

Tanielian, U., & Biau, G. (2021). Approximating lipschitz continuous functions with groupsort neural networks. In *Proceedings of the 24th international conference on artificial intelligence and statistics* (pp. 442–450).

Théate, T., Wehenkel, A., Bolland, A., Louppe, G., & Ernst, D. (2023). Distributional reinforcement learning with unconstrained monotonic neural networks. *Neurocomputing*, *534*, 199–219.

Trindade, J., Vinagre, J., Fernandes, K., Paiva, N., & Jorge, A. (2021). Partially monotonic learning for neural networks. In *International symposium on intelligent data analysis* (pp. 12–23).

Wang, C., Zhang, Y., An, S., & Ding, W. (2025a). An adaptive fuzzy rough neural network and its application in classification. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, *55*(10), 7579–7590.

Wang, J., Qian, Y., Li, F., Liang, J., & Ding, W. (2020). Fusing fuzzy monotonic decision trees. *IEEE Transactions on Fuzzy Systems*, *28*(5), 887–900.

Wang, S., Wang, R., Ren, F., Chen, M., & Yan, R. (2025b). A privacy-preserving federated transfer learning with ship mapping (FTL-SM) framework for accurate ship energy efficiency prediction. *Advanced Engineering Informatics*, *68*, 103569.

Wehenkel, A., & Louppe, G. (2019). Unconstrained monotonic neural networks. In *Advances in neural information processing systems*.

Wei, Y., & Xi, W. (2025). A quasi-monte carlo method based on neural autoregressive flow. *Entropy*, *27*(9), 952.

Wilcoxon, F. (1992). Individual comparisons by ranking methods. In *Breakthroughs in statistics: methodology and distribution* (pp. 196–202).

Wu, X., Fu, D., Li, Z., & Yang, T. (2026). Graph-enhanced & monotonic embeddings: A novel approach to tabular data representation. *Neurocomputing*, *664*, 132108.

Wurm, A. (2023). Complexity of reachability problems in neural networks. In *Reachability problems* (pp. 15–27).

Wurm, A. (2025). Checking extracted rules in neural networks. In *2025 International joint conference on neural networks* (pp. 1–7).

Xu, Z., Yang, Z., Guo, Z., Liu, S., Lin, L., Liu, X., Liu, Y., & Li, H. (2025). Umre: A unified monotonic transformation for ranking ensemble in recommender systems. arXivpreprint arXiv:2508.07613

Yanagisawa, H., Miyaguchi, K., & Katsuki, T. (2022). Hierarchical lattice layer for partially monotone neural networks. In *Advances in neural information processing systems* (pp. 11092–11103).

Yanaka, H., Mineshima, K., Bekki, D., Inui, K., Sekine, S., Abzianidze, L., & Bos, J. (2019). Can neural networks understand monotonicity reasoning? In *Proceedings of the ACL workshop blackboxNLP* (pp. 31–40).

Yang, T., Yan, F., Qiao, F., Wang, J., & Qian, Y. (2025). Fusing monotonic decision tree based on related family. *IEEE Transactions on Knowledge and Data Engineering*, *37*(2), 670–684.

Yao, T., Gao, Y., Zhang, J., Zhang, R., Qian, L., Hu, Q., Wang, X., & Jiang, F. (2025). A stress-dependent strength neural network model for predicting the true triaxial strength of rocks. *Engineering Applications of Artificial Intelligence*, *161*, 112057.

Yeh, I.-C., & Lien, C.-h. (2009). The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert Systems with Applications*, *36*(2), 2473–2480.

You, S., Ding, D., Canini, K., Pfeifer, J., & Gupta, M. (2017). Deep lattice networks and partial monotonic functions. In *Advances in neural information processing systems*.

Zhang, W., & Al Kobaisi, M. (2022). On the monotonicity and positivity of physics-informed neural networks for highly anisotropic diffusion equations. *Energies*, *15*(18), 6823.

Zhao, J., Zhang, H., Wang, Y., Zhai, Y., & Yang, Y. (2024a). Deep isotonic embedding network: A flexible monotonic neural network. *Neural Networks*, *171*, 457–465.

Zhao, X., Wang, L., Zhang, Y., Han, X., Deveci, M., & Parmar, M. (2024b). A review of convolutional neural networks in computer vision. *Artificial Intelligence Review*, *57*(4), 99.

Zhou, Z., Deng, Y., Liu, L., Jiang, H., Peng, Y., Yang, X., Zhao, Y., Ning, H., Allen, N. B., Wilkins, J. T., Liu, K., Lloyd-Jones, D. M., & Zhao, L. (2026). Deep neural network with a smooth monotonic output layer for dynamic risk prediction. *Statistics in Medicine*, *45*(3–5), e70401.

Zhu, B., Ren, S., Weng, Q., & Si, F. (2024). A physics-informed neural network that considers monotonic relationships for predicting NOx emissions from coal-fired boilers. *Fuel*, *364*, 131026.

Zhu, H., Tsang, E. C. C., Wang, X.-Z., & Aamir Raza Ashfaq, R. (2017). Monotonic classification extreme learning machine. *Neurocomputing*, *225*, 205–213.